

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2022 р.

Дипломний проєкт

на здобуття ступеня бакалавра

за освітньо-професійною програмою «Інженерія програмного забезпечення
комп'ютерних систем»

спеціальності «121 Інженерія програмного забезпечення»

на тему: Програмне забезпечення для розпізнавання властивостей та
проблем шкіри людини засобами штучного інтелекту

Виконав студент IV курсу, групи ІТ-81
(шифр групи)

Іовчева Діна Анатоліївна
(прізвище, ім'я, по батькові)

_____ (підпис)

Керівник старший викладач Халус О. А.
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Консультант
з графічної
документації доцент, к.т.н., доц., Ліщук К. І.
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Рецензент професор кафедри ІСТ, д.т.н., Теленик С. Ф.
(посада, науковий ступінь, вчене звання, прізвище та ініціали)

_____ (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ –2022

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
комп'ютерних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ____ ” _____ 2022 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Іовчевій Діні Анатоліївні

(прізвище, ім'я, по батькові)

1. Тема проєкту Програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту

керівник проєкту Халус Олена Андріївна, к.т.н., доцент

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «10» червня 2022 р. №1033-с

2. Термін подання студентом проєкту « 19 » червня 2022 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Аналіз вимог до програмного забезпечення: основні визначення та терміни, опис предметної області, огляд ринку програмних продуктів, постановка задачі.

2) Моделювання та конструювання програмного забезпечення: архітектура програмного забезпечення, математичне забезпечення.

3) Аналіз якості та тестування програмного забезпечення: оцінка якості, види та процес тестування.

4) Впровадження та супровід програмного забезпечення: розгортання програмного забезпечення та робота з програмним забезпеченням.

5. Перелік графічного матеріалу

1) Схема структурна бізнес процесів _____

2) Схема бази даних _____

3) Схема структурна класів програмного забезпечення _____

4) Схема структурна послідовностей _____

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «10» березня 2022 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	10.03.2022	
2	Аналіз існуючих методів розв'язання задачі	20.03.2022	
3	Постановка та формалізація задачі	25.03.2022	
4	Розробка інформаційного забезпечення	30.03.2022	
5	Алгоритмізація задачі	30.03.2022	
6	Обґрунтування вибору використаних технічних засобів	04.04.2022	
7	Розробка програмного забезпечення	04.05.2022	
8	Налагодження програми	24.05.2022	
9	Виконання графічних документів	25.05.2022	
10	Оформлення пояснювальної записки	30.05.2022	
11	Подання ДП на попередній захист	06.06.2022	
12	Подання ДП рецензенту	12.06.2022	
13	Подання ДП на основний захист	19.06.2022	

Студент

(підпис)

Діна ІОВЧЕВА

(ініціали, прізвище)

Керівник

(підпис)

Олена ХАЛУС

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з чотирьох розділів, містить 25 таблиць, 15 рисунків та 13 джерел – загалом 48 сторінок.

Дипломний проєкт присвячений розробці програмного забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту.

Мета роботи - автоматизація створення рекомендацій косметичних засобів на основі зображення обличчя за допомогою методів штучного інтелекту.

Об'єкт дослідження: аналіз стану шкіри засобами штучного інтелекту.

Предмет дослідження: інструмент аналізу стану шкіри.

У розділі аналізу вимог до програмного забезпечення розглянуто загальні положення щодо програмного забезпечення, були проаналізовані існуючі технічні рішення та продукту, визначені варіанти використання програмного забезпечення, сформовані функціональні та нефункціональні вимоги.

Розділ моделювання та конструювання програмного забезпечення присвячений формулюванню і моделюванню основних бізнес-процесів, визначення і опису архітектури та технологій, опису реалізації програмного забезпечення, наведено опис роботи із зображеннями: інструменти, які використовуються для аналізу зображення обличчя.

Розділ аналізу якості програмного забезпечення описує критерії якості програмного забезпечення, визначає функціональність для тестування, план і процес тестування.

Розділ впровадження програмного забезпечення описує спосіб розгортання програмного забезпечення та роботу з ним.

КЛЮЧОВІ СЛОВА: АНАЛІЗ ЗОБРАЖЕННЯ ОБЛИЧЧЯ, РОЗПІЗНАВАННЯ ОБЛИЧЧЯ, РОЗПІЗНАВАННЯ РИС ОБЛИЧЧЯ, ВЕБЗАСТОСУНОК.

ABSTRACT

The explanatory note of the diploma project consists of four sections, contains 25 tables, 15 figures and 13 sources – in total 48 pages.

The purpose of the diploma project is automatization of face features and issues analysis for cosmetics recommendation.

Object of research: skin condition analysis using artificial intelligence methods.

Subject of research: skin condition analysis tool.

The section of the analysis of software requirements contains the general provisions on the software, analysis of modern products and state-of-the-art, setting the use cases, functional and non-functional requirements.

The section of software modeling and design is devoted to formulation and modeling main business processes, setting and description of the software architecture and used technologies, specification of image processing tools.

The software quality analysis and testing section describes the criteria of quality of software, defines the functionality to be tested, test cases and testing processes.

The software implementation and maintenance section describes how to deploy and work with software.

KEYWORDS: FACE IMAGE ANALYSIS, FACE RECOGNITION, FACE FEATURES RECOGNITION, WEB APPLICATION.

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ РОЗПІЗНАВАННЯ
ВЛАСТИВОСТЕЙ ТА ПРОБЛЕМ ШКІРИ ЛЮДИНИ ЗАСОБАМИ
ШТУЧНОГО ІНТЕЛЕКТУ**

Технічне завдання

КПІ.IT-8109.045440.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Діна ІОВЧЕВА

Київ – 2022

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2 ПІДСТАВА ДЛЯ РОЗРОБКИ	4
3 ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1 Вимоги до функціональних характеристик	6
4.2 Вимоги до надійності	7
4.3 Вимоги до складу і параметрів технічних засобів.....	7
4.4 Вимоги до інформаційної та програмної сумісності.....	8
4.5 Вимоги до маркування та пакування	8
4.6 Вимоги до транспортування та зберігання.....	8
4.7 Спеціальні вимоги.....	8
5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	9
6 СТАДІЇ І ЕТАПИ РОЗРОБКИ.....	10
7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	11

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту.

Галузь застосування:

Наведене технічне завдання поширюється на розробку програмного забезпечення “Dermalab” КПІ.ІТ-8109.045440.03.13, котра використовується для автоматизації аналізу шкіри на базі технологій штучного інтелекту та призначена для автоматизації аналізу шкіри та створенню рекомендацій косметичних засобів.

					КПІ.ІТ-8109.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки програмного забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту є завдання на дипломне проектування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

					КПІ.ІТ-8109.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для автоматизації аналізу шкіри, визначення властивостей та проблем шкіри людини засобами штучного інтелекту.

Метою розробки є автоматизація аналізу шкіри для створення рекомендацій косметичних засобів.

					КПІ.ІТ-8109.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Система для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту (далі - система) повинна надавати можливості для аналізу шкіри за допомогою фото, а також рекомендацій косметичних засобів на основі проведеного аналізу.

4.1.1 Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1.1 Для користувача:

- функція перегляду всіх косметичних засобів;
- функція перегляду інформації про косметичний засіб;
- функція реєстрації, авторизації та виходу з системи;
- функція зміни даних користувача;
- функція завантаження зображення у систему для аналізу;
- функція аналізу зображення;
- функція перегляду результатів аналізу зображення: інформації про стан шкіри та рекомендацій косметичних засобів;
- функція відправлення результатів аналізу на електронну пошту;
- функція збереження результатів зображення для зареєстрованого користувача (за електронною адресою)
- функція перегляду минулих результатів аналізу для зареєстрованого користувача.

4.1.2 Розробку виконати на будь-якій платформі (вебзастосунок)

4.1.3 Додаткові вимоги:

- клієнтський застосунок.

4.2 Вимоги до надійності

4.2.1 Передбачити контроль введення інформації.

Система повинна забезпечувати контроль введення інформації для захисту даних сайту від різних типів загроз.

4.2.2 Передбачити захист від некоректних дій користувача.

Система повинна передбачити захист від некоректних дій користувача шляхом валідації даних перед тим, як вони будуть оброблені. Валідація повинна відбуватись у клієнтському застосунку, на серверній частині, а також у базі даних.

4.2.3 Забезпечити захист інформації.

Система працює з конфіденційними даними, тому необхідно забезпечити захист і контроль доступу до даних. Дані про користувача, його фотографії та результати аналізу повинні бути доступним лише самому користувачу.

4.2.4 Забезпечити безперебійну роботу та відновлення після збоїв

Система повинна безперебійно працювати та забезпечувати відновлення роботи у непередбачених випадках збою системи. Якщо система не може обробити запит користувача повинно надсилатись повідомлення про помилку. Збої роботи системи не повинні порушувати цілісність даних або бути вразливим місцем.

4.3 Вимоги до складу і параметрів технічних засобів

4.3.1 Для запуску та функціонування системи повинен бути наявний та сконфігурований сервер із наступними вимогами:

- оперативна пам'ять - не менше 2 Гб;
- жорсткий диск - 50 Гб;
- процесор - чотирьохядерний процесор з частотою 2.1 Гц;
- операційна система - Linux, Ubuntu 20.04 і вище;
- Docker та docker-compose;

- вихід до мережі інтернет, можливість відправляти HTTP-запити.

4.4 Вимоги до інформаційної та програмної сумісності

4.4.1 Програмне забезпечення повинно працювати як веб-застосунок, тому обраним стандартом є HTTPS.

4.4.2 Вхідні дані повинні бути представлені в наступному форматі: HTTP Request.

4.4.3 Результати повинні бути представлені в наступному форматі: HTTP Response.

4.5 Вимоги до маркування та пакування

Вимоги до маркування та пакування не пред'являються.

4.6 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не пред'являються.

4.7 Спеціальні вимоги

Згенерувати установчу версію програмного забезпечення.

					КПІ.ІТ-8109.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		8

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

5.2 Програмне забезпечення повинно мати довідкову систему.

5.3 У склад супроводжувальної документації повинні входити наступні документи:

5.3.1 Пояснювальна записка не менше ніж на 50 аркушах формату А4 (без додатків 5.3.2 - 5.3.6).

5.3.2 Технічне завдання.

5.3.3 Керівництво користувача.

5.3.4 Програма та методика тестування

5.4 Графічна частина повинна бути виконана не менше ніж на 3 аркушах формату А3, котрі включаються у якості додатків до пояснювальної записки:

5.4.1 Схема структура бізнес процесів.

5.4.2 Схема бази даних.

5.4.3 Схема структурна класів програмного забезпечення.

5.4.4 Схема структурна послідовності.

					КПІ.ІТ-8109.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		9

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою проекту	10.03	
2.	Розробка технічного завдання	20.03	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	25.03	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	30.03	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів, схема алгоритму)
5.	Розробка програмного забезпечення: реалізація сервісу для аналізу зображення	17.04	Модель, частина програмного забезпечення для аналізу зображення
6.	Розробка програмного забезпечення	04.05	Програмне забезпечення
7.	Тестування програмного забезпечення	24.05	Тести, результати тестування
8.	Розробка матеріалів графічної частини проекту	25.05	Графічний матеріал проекту
9.	Розробка матеріалів текстової частини проекту	30.05	Пояснювальна записка
10.	Оформлення технічної документації проекту	03.06	Технічна документація

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування

					КПІ.ІТ-8109.045440.01.91	Арк.
						11
Змін.	Арк.	№ докум.	Підп.	Дата.		

**Пояснювальна записка
до дипломного проєкту**

на тему: Програмне забезпечення для розпізнавання властивостей та проблем
шкіри людини засобами штучного інтелекту

КПІ.IT-8109.045440.02.81

Київ – 2022

ЗМІСТ

ВСТУП.....	4
1 АНАЛІЗ ВИМОГ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	5
1.1 Загальні положення.....	5
1.2 Змістовний опис предметної області	5
1.3 Аналіз успішних ІТ-проектів	6
1.4 Аналіз вимог до програмного забезпечення	11
1.5 Постановка задачі.....	17
1.6 Висновки до розділу	18
2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	19
2.1 Моделювання та аналіз програмного забезпечення	19
2.2 Архітектура програмного забезпечення	19
2.3 Конструювання програмного забезпечення	24
2.4. Робота із зображеннями	34
2.5. Захист даних	39
2.6 Висновки до розділу	41
3 АНАЛІЗ ЯКОСТІ ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	42
3.1 Аналіз якості програмного забезпечення	42
3.2 Опис процесів тестування	42
3.3 Опис контрольного прикладу	42
3.4 Висновок до розділу	46
4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	47
4.1 Розгортання розгортання програмного забезпечення	47
4.2 Робота із програмним забезпеченням	47
4.3 Висновки до розділу	47
ВИСНОВКИ.....	48
ВИКОРИСТАНІ ДЖЕРЕЛА	49

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

БД	База даних
ORM	Object-Relational Mapping
API	Application Programming Interface
REST	Representational State Transfer
AWS	Amazon Web Services

					КПІ.ІТ-8109.045440.02.81	Арк.
						3
Змін.	Арк.	№ докум.	Підп.	Дата.		

ВСТУП

Розвиток науки і технологій впливає на все життя, в тому числі й на такі побутові сфери як індустрія краси, сервісів доставки тощо. Всесвітня діджиталізація переносить звичні нам речі, такі як шопінг, у простір інтернету, з'являється купа веб-сайтів та мобільних застосунків, на яких можна замовити будь-який продукт і отримати за декілька годин від кур'єра. Особливого поширення це набуло під час пандемії, коли працювали тільки життєво-необхідні установи, до яких не входили магазини косметики, але попит на неї залишався таким самим. Ця можливість дозволила людям із різних куточків світу замовляти косметику, яку вони не можуть купити у фізичному магазині, і не обмежуватись лише продукцією цього магазину. Водночас, покупці більше не можуть звернутися для консультантів за допомогою під час вибору засобів, адже асортимент таких онлайн-магазинів зазвичай налічує дуже велику кількість товарів і підібрати собі необхідний стає проблемою. У той же розвиток транспорту і глобалізація відкрила не тільки локальні інтернет-ринки, а й глобальні. Під впливом різних медійних особистостей покупці можуть обирати якісні продукти від конкретних брендів, які не представлені на локальному ринку, але за допомогою сервісів доставки можуть замовити їх до своєї квартири. Проблеми вибору косметичного засобу залишаються такими самими - неможливо прийти до спеціаліста або консультанта та отримати інформацію про використання конкретного засобу.

					КПІ.IT-8109.045440.02.81	Арк.
						4
Змін.	Арк.	№ докум.	Підп.	Дата.		

1 АНАЛІЗ ВИМОГ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

1.1 Загальні положення

Першим кроком у розробці програмного забезпечення є визначення задачі, мети, функціональних і нефункціональних вимог, аналіз ринку та успішних проєктів, аналіз існуючих публічних технічних рішень та досліджень. Цей крок допоможе визначити необхідні складові продукту ще до початку його розробки, що формалізує та визначить основні ідеї застосунку. Завдяки цьому шагу буде зменшена кількість неочікуваних змін основної функціональності під час реалізації, програмного забезпечення буде послідовною, за визначеними заздалегідь технологіями, архітектурою, форматами даних та варіантами використання.

1.2 Змістовний опис предметної області

Застосунок міститиме реалізацію системи, яка буде складатися з декількох частин. Система буде поділена на два сервіси: перший відповідатиме за публічний інтерфейс - API (Application Programming Interface) [1], роботу із зберіганням інформації про користувачів, косметичні засоби, критерії тощо, а другий - за роботу із зображеннями та їх аналізом. Для зручності демонстрації додатково буде розроблено графічний інтерфейс для роботи із системою у вигляді веб-сайту.

Перша частина - це сервіс, який буде слугувати каталогом для керування користувачами, косметичними засобами, та даними, які будуть доступні кінцевим користувачам, зберігання зображень. Сервіс буде реалізовувати керування інформацією і визначатиме доступну взаємодію із кінцевим користувачем. Цей сервіс буде надавати інтерфейс у вигляді API. Однією із задач сервісу і є правильний розподіл косметичних засобів відповідно до визначених критеріїв для аналізу зображення обличчя - вони мають бути чітко визначеними і сталими.

Друга частина - сервіс, який буде реалізовувати обробку зображення та його аналіз, алгоритм рекомендації косметичних засобів. Алгоритм роботи сервісу

					КПІ.IT-8109.045440.02.81	Арк.
						5
Змін.	Арк.	№ докум.	Підп.	Дата.		

наступний: сервіс отримує зображення обличчя, передає його на вхід до моделі, яка в свою чергу аналізує зображення, визначає основні риси обличчя, робить їхній опис. Далі за алгоритмом визначаються основні особливості людини, наприклад, почервоніння, або їхня відсутність. Відповідно до результатів аналізу визначаються косметичні засоби, які можуть бути рекомендовані користувачу. Сервіс повертає цей перелік у відповідь на запит. Сервіс буде приховано від сторонніх користувачів, тобто єдина взаємодія відбуватиметься лише з першим сервісом.

1.3 Аналіз успішних ІТ-проектів

1.3.1 Аналіз відомих технічних рішень

Важлива частина проекту - аналіз обличчя на особливості шкіри для рекомендацій косметичних засобів. Такими особливостями є почервоніння, пігментація, висипання, темні кола під очима тощо. Такий аналіз використовується у комерційних застосунках різних брендів косметики, салонів краси, або може використовуватись для медичних цілей, розробка цих проектів зазвичай потребує ресурсів і надає конкурентну перевагу, тому у вільному доступі рішень саме цієї задачі немає. Задачу розпізнання особливостей шкіри можна розбити на підзадачі, для вирішення яких вже існують алгоритми і програми: розпізнання обличчя, розпізнання окремих частин обличчя (носа, рота, очей, контурів), розпізнання окремих особливостей (наприклад, акне) або аналіз визначених частин обличчя за різними факторами.

Розпізнання обличчя - задача не нова, тому зараз існує багато алгоритмів для знаходження обличчя на фотографії. Загалом вони поділяються на два типи: визначення за зразком і за особливостями. [2] У першому підході вхідне зображення поділяється на сегменти, які скануються і класифікуються як обличчя або ні.

Прикладом першого способу визначення обличчя є метод від К. Гарсії і М. Делакіс із використанням згорткової нейронної мережі [3, 4]. Спеціально навчені

					КПІ.ІТ-8109.045440.02.81	Арк.
						6
Змін.	Арк.	№ докум.	Підп.	Дата.		

фільтри і класифікатори за допомогою прикладів та бутстрепових алгоритмів [4] для покращення точності системи дозволяють визначити обличчя із малою кількістю помилок у 7-8%.

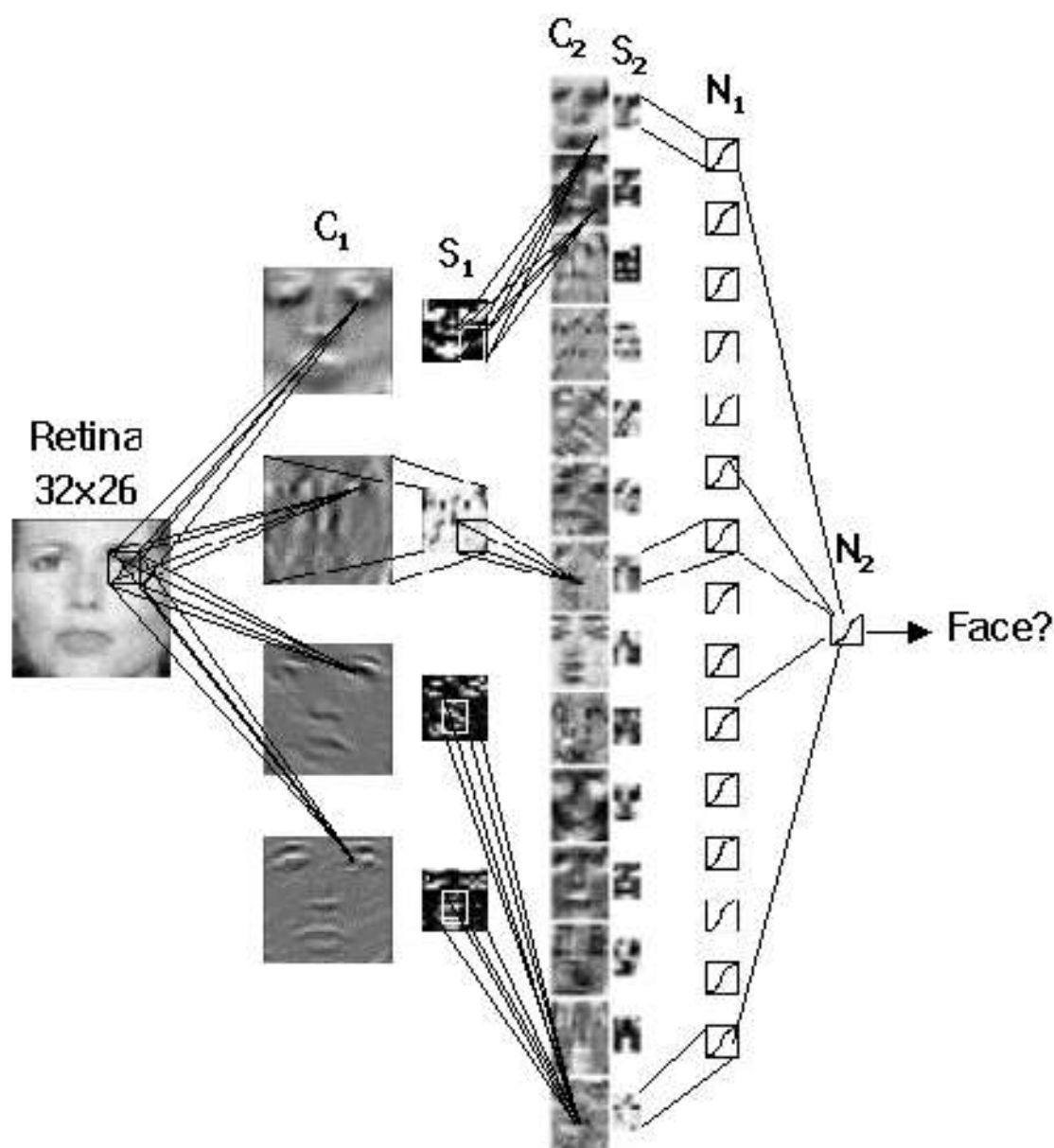


Рисунок 1.1 - Схема роботи системи

Другий підхід полягає у знаходженні частин обличчя або шкіри на зображенні та зменшує кількість потенційних об'єктів. Присутність усіх рис, їхнє розташування та інші показники допомагають визначити, чи є обличчя на рисунку чи ні. Деякі із алгоритмів поєднують розпізнання обличчя за допомогою визначення шкіри і рис обличчя як, наприклад, системи Ш.Х.Ліна, яка спочатку

знаходить пікселі шкіри, а потім локалізує риси обличчя із використанням генетичного алгоритму пошуку. [5]

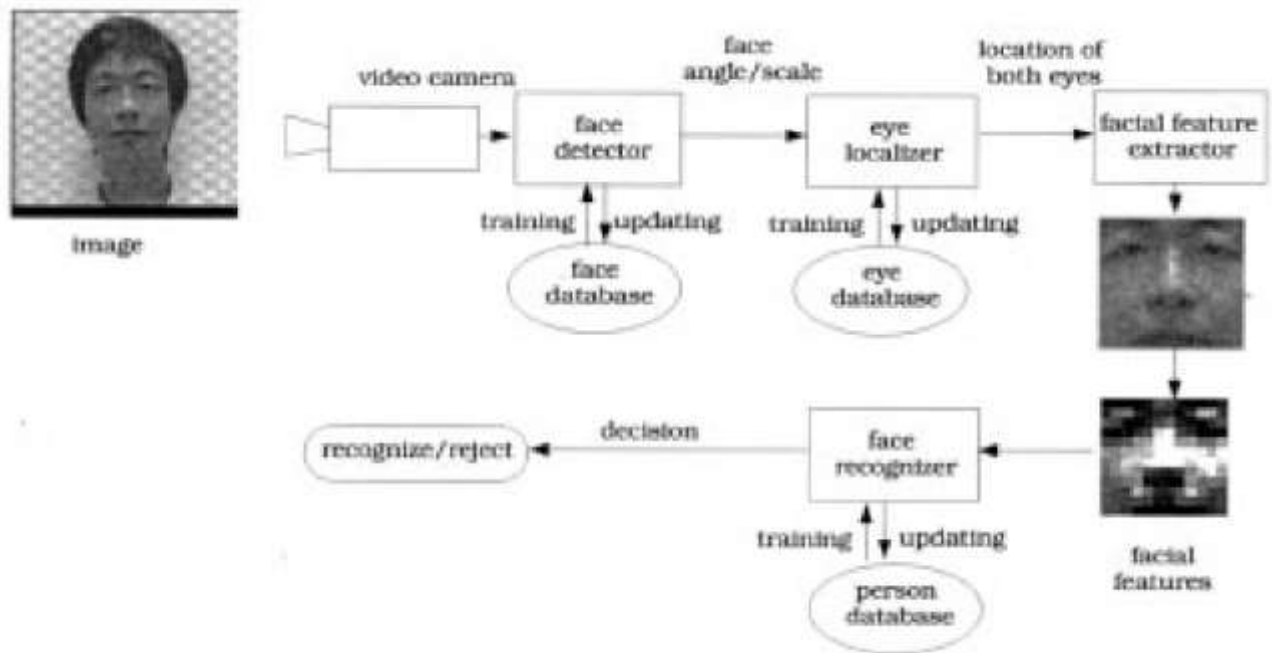


Рисунок 1.2 - Схема роботи алгоритму

Наступним буде розглянуто технічне рішення від Google Machine Learning Kit. Компанія надає систему для знаходження частин обличчя за допомогою ключових точок. Системи визначає контури обличчя і його рис за допомогою 168 точок. У розділі 2.4 буде детально описано такий підхід на прикладі іншої системи. На рисунку 1.3 зображено приклад роботи алгоритму від Google.

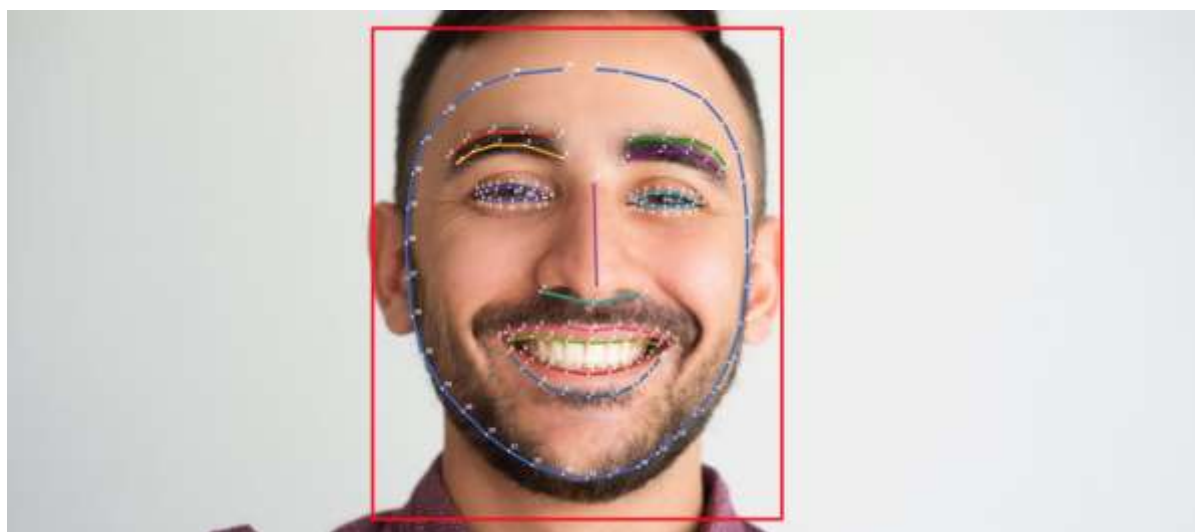


Рисунок 1.3 – Приклад роботи системи від Google

Отже, враховуючи те, що метою проекту буде аналіз особливостей шкіри обличчя, одним із етапів буде визначення саме рис обличчя для створення коректних сегментів для подальшої роботи. Для цього більш вдалим буде алгоритм або система, яка використовує другий підхід, адже він дозволяє не тільки розпізнати обличчя за зразком, а також визначити межі, контури, поділити зображення на частини і мати їхній математичний опис.

1.3.2 Аналіз успішних ІТ-проектів

Застосунки для рекомендації косметичних засобів у результаті аналізу шкіри пов'язані із засобами і брендами косметики, тому зазвичай такі системи існують у якості функції на веб-сайтах брендів косметики.

Одним із таких продуктів є застосунок від бренду dermalogica. Веб-сайт дозволяє завантажити фотографію і отримати інформацію про стан шкіри за такими показниками, як почервоніння, зволоженість тощо. Серед недоліків застосунку можна визначити те, що аналіз не за всіма критеріями є точним, наприклад, фільтр почервоніння дуже чутливий на червоний колір, тому показник часто завищений відносно інших застосунків. Також рекомендації засобів не є комплексними, а підбираються тільки для одного критерію. На рисунку 1.4 зображено приклад аналізу обличчя і рекомендацій засобів за допомогою цього додатку.



Рисунок 1.4 - Приклад аналізу від бренду dermalogica

					КПІ.ІТ-8109.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		9

Іншим популярним додатком для визначення особливостей шкіри і рекомендацій догляду є застосунок від бренду Vichy. Він детально вказує особливості і проблеми шкіри на завантаженому зображенні. Основні проблеми і зони від виділив правильно, чутливість достатня для визначення тільки необхідних областей. Мінусом є також система рекомендацій косметики, оскільки система вказує тільки на декілька конкретних засобів і не надає більшого вибору. Також аналіз можливо зробити лише у браузері мобільного телефону - версії для браузерів або мобільного застосунку немає.

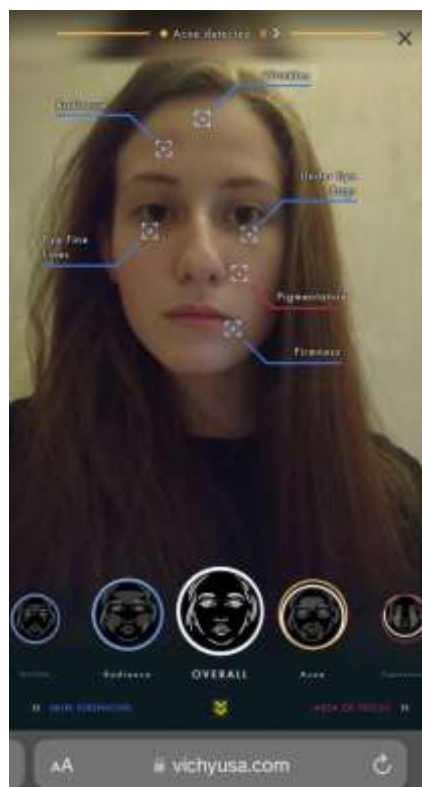


Рисунок 1.5 - Приклад аналізу від бренду vichy

Британський онлайн магазин Superdrug також надає можливість зробити аналіз шкіри за фотографією і отримати перелік косметичних засобів для догляду різних брендів. Рекомендації веб-сайту комплексні і пропонують альтернативи засобам. Перед початком аналізу необхідно пройти тестування і вказати власноруч особливості шкіри, вік, стать, місце проживання, яке визначає вологість, рівень ультрафіолету в регіоні та інші параметри. Ці параметри дозволяють зробити аналіз і рекомендації більш точними. Головним недоліком

					КПІ.IT-8109.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		10

застосунку є вага опитування, яке при різних параметрах і однаковій фотографії надає протилежні результати, наприклад, при збільшенні віку з 20 до 30 років у переліку властивостей шкіри з'являються зморшки і тьмяність шкіри.

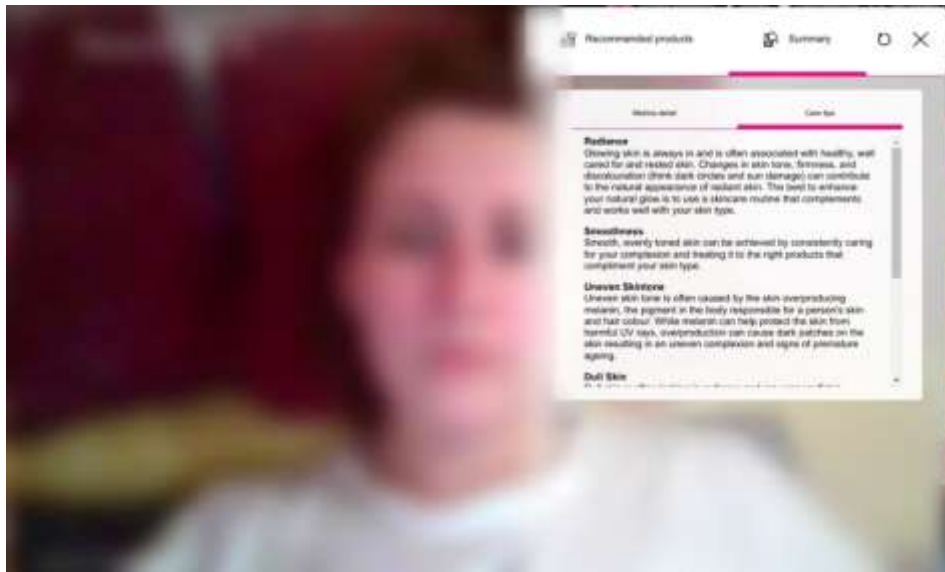


Рисунок 1.6 - Приклад роботи застосунку Superdrug

Отже, можна зробити висновки, що задача аналізу стану шкіри і рекомендації косметичних засобів потребує комплексного підходу, правильного налаштування фільтрів та коефіцієнтів, спиратися не на параметри користувача, а передусім на результати аналізу шкіри, а підбір засобів повинен бути загальним, але не виключним, тобто мати альтернативи.

1.4 Аналіз вимог до програмного забезпечення

Програмне забезпечення призначено для клієнтів - інших застосунків як сторонній сервіс (API), отже важливими вимогами є точність розпізнання особливостей шкіри, відділення сервісу для аналізу зображення і рекомендацій від взаємодії із користувачем.

1.4.1 Розроблення функціональних вимог

Отже, створювана система повинна відповідати таким функціональним вимогам:

					КПІ.IT-8109.045440.02.81	Арк.
						11
Змін.	Арк.	№ докум.	Підп.	Дата.		

- зчитування фотографії;
- розпізнання особливостей шкіри відповідно до характеристик;
- керування даними косметичних засобів;
- керування користувачами застосунку;
- створення переліку рекомендацій косметичних засобів на основі результатів;
- збереження результатів аналізу для користувача.

Для дослідження функціональних вимог більш детально необхідно сформулювати можливі варіанти використання застосунку. Для усіх варіантів використання учасником буде користувач застосунку-клієнта.

Таблиця 1.1 – Варіант використання UC01

Назва	Реєстрація користувача
Опис	Користувач реєструється в системі
Результат	Користувача зареєстровано в системі.
Сценарій	Користувач натискає кнопку “зареєструватися” у застосунку-клієнті, вводить необхідні дані і підтверджує реєстрацію. Якщо дані коректні, то користувача буде зареєстровано в системі.

Таблиця 1.2 – Варіант використання UC02

Назва	Авторизація користувача (логін)
Опис	Користувач авторизується в системі.
Передумови	Користувач повинен бути зареєстрованим.
Результат	Користувача авторизовано в системі.
Сценарій	Користувач тисне кнопку “логін” у застосунку-клієнті, вводить дані і підтверджує логін. Якщо дані коректні, користувача буде авторизовано в системі.

Таблиця 1.3 – Варіант використання UC03

Назва	Вихід із системи (логаут)
Опис	Користувач виходить із системи
Передумови	Користувач повинен бути авторизований у системі
Результат	Користувача вийшов із системи.
Сценарій	Користувач тисне кнопку “вийти із системи, підтверджує дію.

Таблиця 1.4 – Варіант використання UC04

Назва	Перегляд каталогу.
Опис	Користувач має доступ до каталогу косметичних засобів.
Результат	Користувач може побачити косметичні засоби..
Сценарій	Користувач тисне кнопку “каталог”, він перенаправлений на сторінку каталогу.

Таблиця 1.5 – Варіант використання UC05

Назва	Завантаження зображення.
Опис	Користувач завантажує зображення для подальшого аналізу.
Результат	Зображення завантажено і може використовуватися для аналізу.
Сценарій	Користувач тисне кнопку “завантажити зображення”, обирає зображення і підтверджує.

Таблиця 1.6 – Варіант використання UC06

Назва	Аналіз фотографії
Опис	Користувач має можливість зробити аналіз своєї фотографії.
Передумови	Зображення завантажено.
Результат	Проведено аналіз фотографії, доступен його результат та перелік косметичних засобів.
Сценарій	Користувач тисне кнопку “провести аналіз”.

Таблиця 1.7 – Варіант використання UC07

Назва	Перегляд результату аналізу.
Опис	Користувач після аналізу має змогу переглянути результат аналізу шкіри.
Передумови	Аналіз було проведено.
Результат	Користувачеві надається аналіз шкіри.
Сценарій	Користувач тисне кнопку “результат аналізу”.

Таблиця 1.8 – Варіант використання UC08

Назва	Перегляд рекомендацій косметичних засобів
Опис	Користувач після аналізу має змогу переглянути рекомендації косметичних засобів.
Передумови	Аналіз було проведено.
Результат	Користувачеві надаються рекомендації косметичних засобів.
Сценарій	Після проведення аналізу користувач тисне на кнопку “рекомендації”.

Таблиця 1.9 – Варіант використання UC09

Назва	Відправити результат на електронну пошту.
Опис	Користувач після аналізу має змогу відправити результат та рекомендації на електронну пошту. Передумови: аналіз було проведено.
Передумови	Аналіз було проведено.
Результат	Отриманий результат відправлено на пошту.
Сценарій	Після проведення аналізу користувач натискає кнопку “відправити результат на пошту”, вводить свою електронну адресу і тисне кнопку “надіслати”.

Таблиця 1.10 – Варіант використання UC10

Назва	Переглянути результат минулих аналізів.
Опис	Користувач має змогу переглянути результати минулих аналізів
Передумови	Користувач зареєстрований в системі, користувач авторизований, під час проведення минулого аналізу був авторизований у системі, або надіслав результат на пошту, з якою реєструвався в системі.
Результат	Наявність результатів минулих аналізів.
Сценарій	Користувач тисне на кнопку із власним ім'ям, переходить до сторінки профілю, обирає вкладку “попередні результати”.

Далі на рисунку 1.7 надано структурну схему варіантів використання.

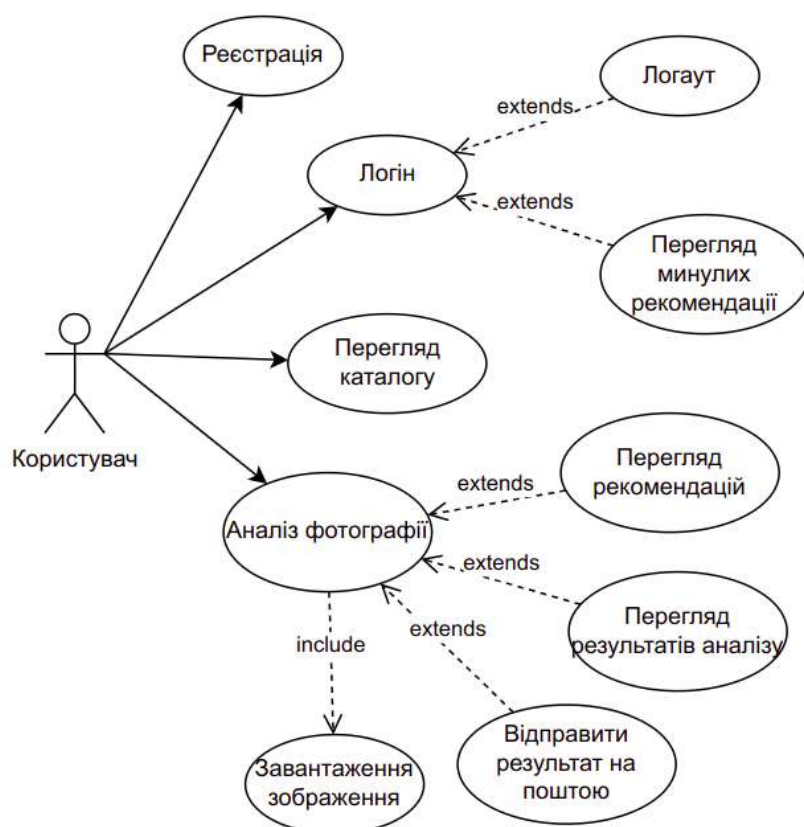


Рисунок 1.7 - Схема структурна варіантів використання

Для перевірки наскільки функціональні вимоги співвідносяться з варіантами використання потрібно створити таблицю для відображення.

Таблиця 1.11 – Таблиця відображення варіантів використання і функціональних вимог

	UC01	UC02	UC03	UC04	UC05	UC06	UC07	UC08	UC09	UC10
Зчитування фотографії										
Розпізнання особливостей шкіри										
Керування косметичним и засобами										
Керування користувачам и										
Перелік рекомендацій										
Збереження результатів										

1.4.2 Розроблення нефункціональних вимог

Застосунок повинен відповідати наступним функціональним вимогам:

- може працювати як самостійний застосунок і як сторонній сервіс (у вигляді API);
- швидко обробляти запити - до 5 секунд для запитів, пов'язаних з обробкою зображення, до 1 секунди для інших запитів.

1.5 Постановка задачі

Актуальність теми: в умовах зростання пандемії та переходу усіх сфер життя у інтернет, інтернет-магазини стають більш потрібними і зручними.

Головною метою дипломної роботи є створення застосунку, для автоматизації створення рекомендацій косметичних засобів на основі зображення обличчя за допомогою методів штучного інтелекту.

Головним завданням розробки є аналіз шкіри на зображенні обличчя і створення рекомендацій косметичних засобів із наданого переліку.

Для досягнення мети необхідно реалізувати наступні задачі:

- створення системи для аналізу шкіри за допомогою штучного інтелекту;
- створення алгоритму рекомендацій косметичних засобів на основі аналізу шкіри;
- ведення користувачів та косметичних засобів.

Практичне значення: розроблено програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту і створення рекомендацій косметичних засобів.

					КПІ.IT-8109.045440.02.81	Арк.
						17
Змін.	Арк.	№ докум.	Підп.	Дата.		

1.6 Висновки до розділу

У даному розділі було проведено аналіз вимог програмного забезпечення. Були досліджена предметна область, а також існуючі технічні рішення і продукти. Відповідно до результатів аналізу застосунків були визначені їхні слабкі і сильні сторони, які полягли в основу визначення функціональних та нефункціональних вимоги. Окрім цього були визначені можливі сценарії використання, вони були детально описані і зображені у схемі мовою UML. Також була створена матриця залежності між вимогами і варіантами використання для того, щоб перевірити, наскільки повно функціональні вимоги враховують варіанти використання застосунку. У результаті було визначено, що продукт виконує визначені вимоги.

					КПІ.IT-8109.045440.02.81	Арк.
						18
Змін.	Арк.	№ докум.	Підп.	Дата.		

2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Моделювання та аналіз програмного забезпечення

Моделювання та аналіз програмного забезпечення є важливим етапом для створення якісного продукту. Моделювання програмного забезпечення можна почати з моделювання бізнес-процесів для того, щоб чітко визначити усі процеси, які будуть у застосунку. Для цього використовують BPMN-діаграму (англ. Business Process Model and Notation).

Отже, програмний продукт надаватиме такі функції користувачі:

- реєстрація та авторизація користувача;
- перегляд каталогу;
- аналіз зображення обличчя;
- перегляд аналізу зображення та рекомендацій косметичних засобів;
- надсилання результатів на пошту.

На схемі бізнес-процесів зображено декілька складних процесів: реєстрація або авторизація користувача, аналіз зображення і надсилання результатів аналізу шкіри і рекомендацій косметичних засобів на пошту. Схема зображена у графічних матеріалах, лист 1.

2.2 Архітектура програмного забезпечення

Визначення архітектури проекту є важливим етапом створення застосунку, оскільки на цьому етапі враховуються усі ризики, які можуть виникнути в подальшому під час розробки, підтримки, оновлення застосунку.

Застосунок складатиметься з двох сервісів: сервісу для керування користувачами, косметичними засобами і взаємодією із іншим сервісом, а також сервісу для аналізу зображення і створення рекомендацій.

					КПІ.IT-8109.045440.02.81	Арк.
						19
Змін.	Арк.	№ докум.	Підп.	Дата.		

Для реалізації було обрано мову програмування Python і фреймворк Flask [6] для першого сервісу і AWS Lambda для другого сервісу. Загальна архітектура цієї частини буде багатошаровою, яка поділятиметься на три шари: шар доступу до даних, шар бізнес логіки, шар інтерфейсу. Клієнт посилає запити до кінцевих точок (endpoints) застосунку, які визначені на шарі інтерфейсу. Далі він взаємодіє із шаром бізнес логіки, викликаючи необхідні йому методи і функції. Останній обробляє дані або залучає до роботи шар доступу до даних за необхідності, який працює безпосередньо із даними і базою даних. Таким чином, кожен шар має конкретну визначену відповідальність. Усі три шари відповідно до принципу декомпозиції можна деталізувати на рівень агрегації класів, що вирішують ту чи іншу задачу, та називаються патерном. В кожному патерні виділяють абстрактну частину, яка впровадить ізоляцію конкретної реалізації від загальної поведінки і імплементацію, яка надає відповідним класам чітко визначені методи, сигнатури методів, відповідну реалізацію.

Обидва сервіси матимуть таку архітектуру, тільки для сервісу роботи зображень клієнтом буде інший сервіс. З точки зору взаємодії для складових проекту обрано підхід REST API, який спирається на “клієнт-серверну” архітектуру. Завдяки цьому підходу клієнтська і серверна частини будуть незалежними один від одної, а їхня комунікація відбуватиметься за допомогою HTTP-запитів та відповідей. Для спілкування визначають 4 основні методи: методи HTTP-запитів: GET (отримати), PUT (додати, замінити), POST (додати, змінити, видалити), DELETE (видалити). Таким чином, дії CRUD (Create-Read-Update-Delete) можуть виконуватися як з усіма 4-ма методами, так і тільки за допомогою GET і POST. Такий підхід забезпечить максимальну гнучкість [1].

Розподіл застосунку на два сервіси потребує вимога до швидкості обробки запитів на аналіз зображення і створення рекомендацій. Обробка зображень моделлю може займати часу більше, ніж 5 секунд, чим може викликати великі черги або навіть помилку очікування. Використання AmazonWebServices дозволяє вирішити цю проблему за допомогою AWS Lambda - швидкого сервісу, який

					КПІ.IT-8109.045440.02.81	Арк.
						20
Змін.	Арк.	№ докум.	Підп.	Дата.		

розрахований на швидку обробку даних. Окрім того, збереження зображень у S3 зменшує час його передачі. [7] Це дозволить суттєво зменшити час виконання запитів, порівняно із реалізацією власного середовища для аналізу зображень.

2.2.1 Зберігання даних

Необхідно визначити основні терміни, які використовуються при роботі із даними:

- сутність - об'єкт, інформація про який зберігається в базі даних;
- атрибут - властивість об'єкта;
- відношення (зв'язок) - взаємодія між декількома сутностями.

Для зберігання більшої кількості даних, таких як інформація про косметичні засоби, категорії, користувача тощо буде використана база даних із СУБД PostgreSQL. [8] Відсутність однозначної структури даних деяких сутностей, таких як аналіз, який повинен містити інформацію про аналіз фотографії, користувача і додаткові дані, вимагає використання NoSQL бази даних dynamodb від AmazonWebServices.[7,8] Для зберігання і доступу до зображень буде використовуватись сервіс S3 Bucket від AmazonWebServices. У графічних матеріалах зображено схема реляційної бази даних, лист 2.

Далі у таблиці буде наведено опис сутностей і атрибутів.

Таблиця 2.1 - Опис сутностей і атрибутів реляційної бази даних

№	Назва сутності	Атрибут	Тип	Призначення
1	User	id	int	Ідентифікатор
		uuid	varchar	Рядковий ідентифікатор, використовується для того, щоб не надавати інформацію про кількість даних у таблиці.
		user_email	varchar	Пошта користувача, унікальне поле
		user_firstname	varchar	Ім'я користувача

Продовження таблиці 2.1

№	Назва сутності	Атрибут	Тип	Призначення
		user_lastname	varchar	Прізвище користувача
		password	varchar	Пароль, зберігається у зашифрованому вигляді
		created_at	timestamp	Службова інформація про дату створення
		updated_at	timestamp	Службова інформація про дату оновлення
2	Functionality	id	int	Ідентифікатор
		uuid	varchar	Рядковий ідентифікатор, використовується для того, щоб не надавати інформацію про кількість даних у таблиці.
		title	varchar	Назва функцій
		created_at	timestamp	Службова інформація про дату створення
		updated_at	timestamp	Службова інформація про дату оновлення
3	Functionality_product_map	id	int	Ідентифікатор. Один, оскільки дані таблиці не будуть передаватись клієнтам.
		functionality_id	int	Зовнішній ключ до таблиці Functionality
		product_id	int	Зовнішній ключ до таблиці Product
4	Product	id	int	Ідентифікатор
		uuid	varchar	Рядковий ідентифікатор
		product_name	varchar	Назва засобу
		desription	text	Опис
		size	int	Розмір (у мл)
		ingredients	text	Склад
		picture_url	varchar	Посилання на зображення
		category_id	int	Зовнішній ключ, категорія продукту
		created_at	timestamp	Службова інформація про час і дату створення
		updated_at	timestamp	Службова інформація про час і дату оновлення

Продовження таблиці 2.1

5	Category	id	int	Ідентифікатор
		uuid	varchar	Рядковий ідентифікатор
		title	varchar	Назва категорії
		category_type	type, enum	Тип категорії, окремий тип даних в PostgreSQL – enum.
		created_at	timestamp	Службова інформація про час і дату створення
		updated_at	timestamp	Службова інформація про час і дату оновлення
6	Analysis	id	int	Ідентифікатор
		uuid	varchar	Рядковий ідентифікатор
		category_type	type, enum	Тип категорії
		user_id	int	Зовнішній ключ, користувач
		picture_url	varchar	Посилання на зображення
		created_at	timestamp	Службова інформація про час і дату створення
		updated_at	timestamp	Службова інформація про час і дату оновлення
7	Analysis_product_map	id	int	Ідентифікатор
		analysis_id	int	Зовнішній ключ, ідентифікатор аналізу
		product_id	int	Зовнішній ключ, ідентифікатор продукту

Для нереляційної бази даних побудувати таку таблицю немає можливості, оскільки особливість NoSql баз даних полягає у тому, що структура даних попередньо невідома або може змінюватись. Основними атрибутами, які будуть у таблиці бази даних, будуть ключі (первинний та вторинний), ідентифікатори користувача, аналізу із реляційної бази даних, перелік продуктів, а також перелік критеріїв аналізу із різними характеристиками залежно від критерію, такі як назва, ступінь, відсоток, кількість тощо.

2.3 Конструювання програмного забезпечення

Застосунок поділено на два сервіси, які спілкуються за допомогою HTTP-запитів. Сервіс для роботи з інформацією про користувачів та засоби косметики містить стандартну архітектуру: класи-моделі даних, які відповідають сутностям у базі даних, класи-сервіси, які реалізують взаємодію із базою даних, класи для бізнес логіки, класи і функції, що визначають інтерфейс взаємодії із системою.

На листі 3 зображено структурну схему класів першого сервісу.

Шар доступу до даних містить класи, які повністю відповідають сутностям у базі даних. Шар сервісів містить клас Service, у якому реалізовані усі спільні методи для роботи із даними: дістати записи, дістати записи за фільтрами та критеріями, зберегти, оновити, створити тощо. У якості ORM(Object-Relational Mapping) використовується SQLAlchemy, бібліотека python, яка надає широкі можливості для роботи із базами даних.

Таблиця 2.2 - Основні елементи сервісу для роботи із системою

№	Назва	Тип елемента	Характеристика
1.	User	Клас	Користувач системи, до якого прив'язані результати аналізу шкіри. Містить дані користувача, а також деякі методи для відображення даних. Успадковується від класу ORM Model для співставлення із сутністю бази даних.
2.	Product	Клас	Косметичний засіб. Успадковується від класу ORM Model для співставлення із сутністю бази даних.
3.	Category	Клас	Категорія косметичних засобів. Успадковується від класу ORM Model для співставлення із сутністю бази даних. Містить у собі поле category_type, яке є переліченням типів косметики - декоративна або доглядова.
4.	CategoryType	Перелічення	Enum, перелічення. Містить у собі два поля, які вказують на назви типів косметики: makeup, skincare.

Продовження таблиці 2.2

№	Назва	Тип елемента	Характеристика
5.	Functionality	Клас	Призначення косметики. Використано у алгоритмі створення рекомендацій косметики. Може бути використаним для фільтрації продуктів, тощо. Успадковується від класу ORM Model.
6.	Analysis	Клас	Результат аналізу. Містить дані про аналіз шкіри, користувача, посилання на зображення. Буде містити іншу частину інформації, формат якої може змінюватись, у NoSql базі даних. Успадковується від класу ORM Model.
7.	Service	Клас	Сервіс для роботи із моделями і сесією бази даних. Містить багато методів для роботи із базою даних: дістати всі сутності, конкретну сутність, створити, видалити, дістати деякі поля сутності тощо.
8.	UserService	Клас	Сервіс для роботи із користувачами. Успадковується від класу Service.
9.	ProductService	Клас	Сервіс для роботи із косметичними засобами. Успадковується від класу Service.
10.	CategoryService	Клас	Сервіс для роботи із категоріями. Успадковується від класу Service.
11.	AnalysisService	Клас	Сервіс для роботи із таблицею результатів аналізів. Також взаємодіє із сервісом NoSql бази даних і сервісом для роботи з AWS. Успадковується від класу Service.
12.	PictureService	Клас	Сервіс для роботи із зображеннями. Відповідає за збереження зображень, правильну обробку та найменування.
13.	AWSService	Клас	Сервіс для взаємодії з AWS Lambda. Встановлює зв'язок, формує запити і відповіді.
14.	UserApi	Клас	Клас, який реалізує методи для взаємодії із клієнтом. Клас відповідає за роботу із користувачами. Успадковується від класу MethodApi фреймворку, який реалізує усі методи для роботи із системою. Є кінцевою точкою.

Продовження таблиці 2.2

№	Назва	Тип елемента	Характеристика
15.	ProductApi	Клас	Клас, який реалізує методи для взаємодії із клієнтами. Клас відповідає за роботу із косметичними засобами. Успадковується від класу MethodApi фреймворку Flask, який реалізує усі необхідні методи для роботи із системою. Реєструється як шлях - кінцева точка.
16.	ImageApi	Клас	Клас, який реалізує методи для взаємодії із клієнтами. Клас відповідає за роботу із зображеннями. Успадковується від класу MethodApi фреймворку Flask, який реалізує усі необхідні методи для роботи із системою. Реєструється як шлях - кінцева точка.
17.	AnalysisApi	Клас	Клас, який реалізує методи для взаємодії із клієнтами. Клас відповідає за створення запиту на аналіз та отримання минулих результатів аналізу. Успадковується від класу MethodApi фреймворку Flask, який реалізує усі необхідні методи для роботи із системою. Є кінцевою точкою.
18.	login	Функція	Функція для обробки запиту на вхід в систему користувача. Реєструється як шлях - кінцева точка.
19.	logout	Функція	Функція для обробки запиту на вихід із системи користувача. Реєструється як шлях - кінцева точка.
20.	register	Функція	Функція для обробки запиту на реєстрацію користувача. Реєструється як шлях - кінцева точка.
21.	auth	Функція	Функція, яка відповідає за реєстрацію користувача.
22.	decorators	Пакет	Пакет, який містить необхідні декоратори для функцій та методів. Використовуються для того, щоб винести одну й ту саму дію (наприклад, перевірку на авторизацію) в окрему функцію і використати як декоратор.

Продовження таблиці 2.2

№	Назва	Тип елемента	Характеристика
23.	Schemas	Пакет	Пакет, який містить схеми формату даних різних класів та сутностей. Використовуються як валідація даних, що приходять із запитом користувача, а також для формування відповіді у певному форматі. Викидає помилку, якщо формат даних некоректний.

Сервіс для роботи із зображеннями схожу архітектуру: модуль для взаємодії із базою даних dynamodb, модуль layers, який реалізує бізнес логіку, модуль serverless, який визначає інтерфейс взаємодії із системою. На рисунку 2.1 зображено основні пакети і класи сервісу.

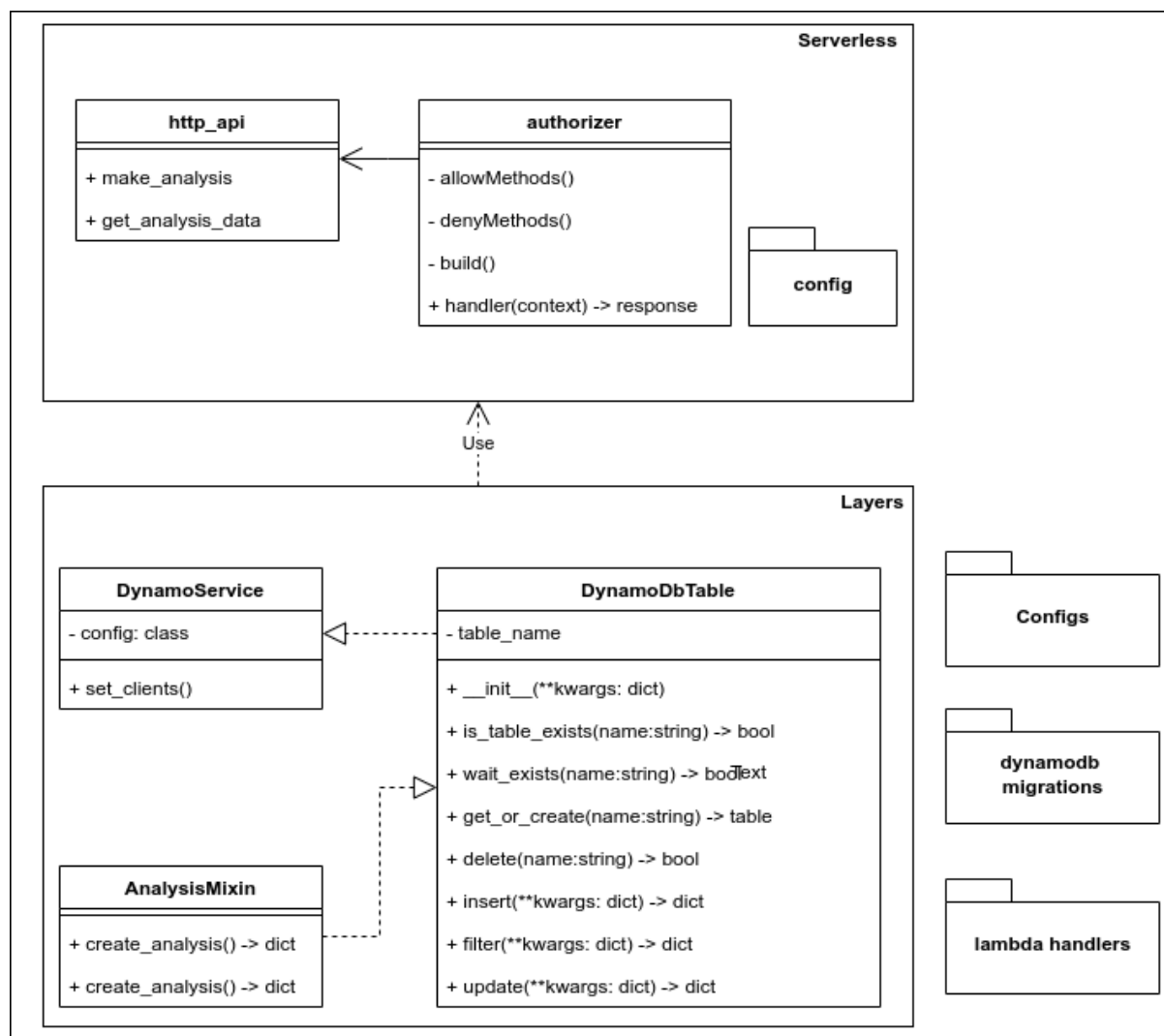


Рисунок 2.1 – Схема структурна класів і пакетів сервісу для аналізу

Пакети в python - директорії, які містять у собі файли python, різні модулі або інші пакети. Таким чином, цей шар ділиться на 4 пакети: lambda-handlers, dynamodb migrations, layers, serverless. Кожен з них містить різні конфігураційні файли для налаштування роботи AwsLambda. У пакеті lambda handlers знаходяться файли, які вказує на поведінку функції, коли до неї приходить запит. У пакеті dynamo migrations знаходяться файли для конфігурації NoSQL бази даних dynamodb. Пакет layers містить інформацію та класи, які повинні бути сталими і розділятися між декількома функціями. Також саме у цьому шарі зберігаються різні константи, enums, службові функції, допоміжні функції. У пакеті serverless визначаються безпосередньо функціями AwsLambda. У застосунку будуть використовуватися дві функції: одна для авторизації користувача (authorizer), а друга - для аналізу зображення. Вони надають перелік кінцевих точок (endpoints), які можуть бути викликані сервісами. Далі у таблиці 2.3 наведено детальний опис функцій і класів цього сервісу.

Таблиця 2.3 – Основні елементи сервісу для роботи із зображеннями

№	Назва	Тип елементу	Характеристика
1	DynamoDbTable	Клас	Клас, який відповідає за роботу з NoSql базою даних. Основна взаємодія буде відбуватися саме у цьому сервісі, тому необхідно визначити клас, який би встановлював з'єднання із базою даних, налаштовував коректний ввід та вивід даних, зберігання, видалення, створення елементів. Також цей клас вказує формат даних, який буде передаватись у базу даних. Успадковується від DynamoService.

Продовження таблиці 2.3

2	DynamoService	Клас	Клас, який використовується для конфігурації NoSql бази даних DynamoDb.
3	AnalysisMixin	Клас	Клас, який визначає роботу із даними аналізів. У цьому класі визначаються необхідні поля для додавання або видалення даних, робота із аналізом.
4	Serverless.config	Пакет	Містить конфігураційні файли для взаємодії із AWS Lambda. Налаштування вказані і для Lambda безпосередньо, так і для AWS Gateway.
5	authorizer	Клас	Містить конфігураційні дані для авторизації користувача. Містить методи для роботи із дозволами. Формує дані сесії для передачі їх у інші методи.
6	http_api	Клас	Визначає методи, з якими може взаємодіяти клієнт. Запит спочатку оброблюється методами класу authorizer, який в свою чергу передає необхідні дані про користувача.

AWS Lambda буде використовуватись разом із Amazon Api Gateway. Gateway надає засоби для маршрутизації усіх запитів до системи, моніторингу, конфігурації та безпеки. Саме за допомогою нього клієнти можуть отримати доступ до даних і логіки сервісу. У проекті буде реалізовано RESTful API, але сервіс також дозволяє налаштувати комунікацію за допомогою сокетів.

Також у сервісі буде використовуватись сховище Amazon S3 – Simple Storage Service. Цей сервіс дозволяє досить просто зберігати дані у різних розмірах, що є гарним рішенням для фотографій, оскільки вони можуть займати багато місця.

Також у системі будуть виділені допоміжні сервіси, які виконуватимуть службову роботу. Такими сервісами буде сервіс для роботи із Redis (сховище даних формату ключ-значення) і сервіс для роботи із Celery (асинхронною чергою завдань) [6].

Сервіс для роботи із Redis буде виконувати одну функцію – зберігання сесії користувача для того, щоб вона була доступна і однакова як в сервісі у AWS, так і в застосунку на Flask. Для цього буде встановлено і налаштовано з'єднання до Redis з обох сервісів, а допоміжний сервіс буде тільки робити функцію запису та читання даних за ключем.

Сервіс для роботи із Celery буде використовуватись для відправки результатів аналізів поштою. Формування зображення листа буде покладено на сервіс роботи із каталогами, а celery буде реалізовувати саме інтерфейс для роботи із чергою і її виконанням.

На листі 4 графічних матеріалів зображено структурну схему послідності роботи застосунку для бізнес-процесу авторизації, завантаженню фотографії і аналізу зображення.

Для демонстрації роботи програмного забезпечення буде зроблена клієнтська частина за стосунку із зручним користувацьким інтерфейсом. Вона є незалежною від програмної, а взаємодія відбуватиметься шляхом запитів на відповідні URL-адреси і отримання відповідей. Для реалізації було використано бібліотеку React, оскільки вона є однією із найбільш популярних фреймворків і бібліотек JavaScript. Проект має досить просту і зрозумілу архітектуру: усі файли розташовуються в директорії src/ і розподіляються по каталогам відповідно

					КПІ.IT-8109.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		30

призначенню. Каталоги містять файли-компоненти, які складають ієрархічну систему. Також проект містить налаштування шляхів, спільні файли для констант і сервісів для взаємодії із серверною частиною і повторному використанню коду. У таблиці 2.4 наведено основні URL адреси і запити, які можна до них робити.

Таблиця 2.4 – Основні адреси програмного забезпечення

№	Адреса	Метод	Запит	Відповідь
1.	products/	GET	Запит на отримання переліку усіх косметичних засобів.	Перелік косметичних засобів
		POST	Запит на створення косметичного запиту. Дозволений лише суперкористувачам	Створений косметичний засіб
2.	products/categories	GET	Запит на отримання косметичних засобів за категорією.	Косметичних засобів за категорією.
3.	user/login	POST	Запит на вхід у систему. Містить у тілі дані користувача: електронну пошту і пароль	Дані користувача, створюється нова сесія.
4.	user/register	POST	Запит на реєстрацію користувача. Містить у тілі запиту дані користувача: ім'я, прізвище, пошту, номер телефону, два паролі.	Дані користувача, створюється нова сесія.
5.	user/logout	POST	Вихід із системи	-

Продовження таблиці 2.4

№	Адреса	Метод	Запит	Відповідь
6.	user/profile	GET	Запит на отримання інформації про користувача. Не містить у собі параметрів, користувач визначається із сесії	Дані про користувача, якщо він авторизований.
	user/profile	POST	Запит на зміну даних користувача. Містить у тілі нові дані користувача.	Повертає результат – успіх і дані або помилку.
7.	user/profile/analysis	GET	Запит на перегляд минулих результатів аналізу. Не містить у собі параметрів, оскільки користувач визначається із сесії	Повертає дані про результати аналіз і перелік косметичних засобів
8.	images	POST	Запит на завантаження фотографії. У тілі містить blob-дані про фотографію.	Повертає дані про фотографію: url знаходження.
9.	analysis/	POST	Запит на проведення аналізу. У тілі вказується адреса фотографії. Дані про користувача вказуються із сесії	Результату аналізу: загальний стан шкіри по категоріях, перелік косметичних засобів за категоріями, посилання на результуючі зображення.

Продовження таблиці 2.4

№	Адреса	Метод	Запит	Відповідь
10.	analysis/uuid/send	POST	Запит на відправлення аналізу на пошту користувачу. Містить uuid аналізу у запиті, може містити пошту користувача.	Результат – успіх або помилка. Система окремо відправить поштою листа.

Бібліотека React використовується разом із бібліотекою Redux для пришвидшення роботи і динамічної зміни клієнтського контенту. Redux керує станом програми, редагує живий код. Основна його концепція полягає у трьох складових: єдине джерело істини, стан «тільки для читання» і зміна за допомогою чистих функцій. Єдине джерело істини говорить про те, що стан за стосунку зберігається в дереві об'єктів в одному сховищі. Стан «тільки для читання» вказує обмеженість способів зміни стану: усі зміни централізовані, зовнішні об'єкти стан не змінюють, а лише виражають намір це зробити. Зміна відбувається за допомогою спеціальних функцій, які приймають попередній стан і дію, і повертають наступний стан. Схема роботи зображена на рисунку 2.2.

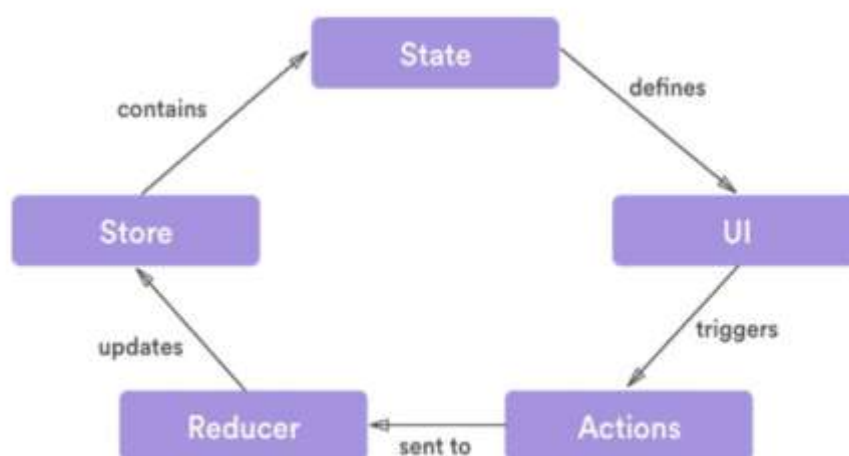


Рисунок 2.2 – Схема роботи Redux

2.4. Робота із зображеннями

Робота із зображеннями у програмному забезпеченні полягає у розпізнаванні обличчя, визначення основних рис обличчя, частин, аналізу цих частин за такими показниками, як почервоніння, пігментація, темні кола під очима тощо.

Першим кроком є розпізнавання і визначення рис обличчя. У розділі 1.3 було розглянуто декілька методів і обрано підхід, який використовується для визначення рис обличчя на зображенні. Для цього програмного забезпечення було обрано поширену модель, яка заснована на деревах рішень.[9] Вона була натренована на наборі даних, який містив у собі 68 контрольних точок: контури носа, рота, очей, підборіддя. Кількість точок залежить від набору даних, на якому навчається модель. Детектор використовує гістограму напрямлених градієнтів та метод опорних векторів для розпізнавання обличчя на завантаженому зображенні, а також за допомогою контрольних точок визначаються риси обличчя.

Метод опорних векторів - алгоритм, який використовується для класифікації даних, тобто поділу на класи. Робота класифікатору полягає у тому, що він знаходить рівняння, яке поділяє вхідний набір даних на два класи найбільш оптимальним шляхом. Рівняння також містить ваги для різних параметрів, які визначаються під час навчання таким чином, щоб об'єкти класів знаходились найбільш далеко від лінії розподілу. Об'єкти, за допомогою визначається ця лінія, і називаються опорними векторами. Максимізація відстані між опорними векторами дозволяє зменшити кількість помилок. На рисунку 2.3 зображена робота алгоритму. [10, 11]

					КПІ.IT-8109.045440.02.81	Арк.
						34
Змін.	Арк.	№ докум.	Підп.	Дата.		

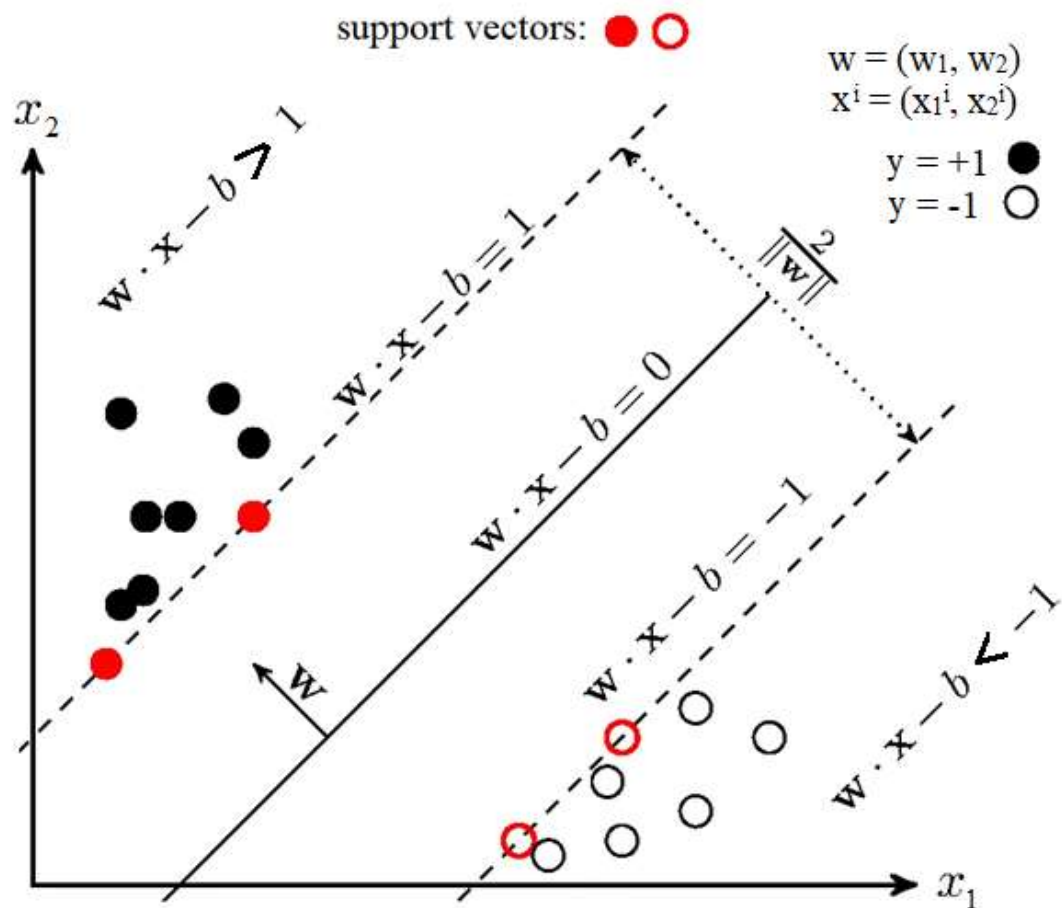


Рисунок 2.3 - Робота алгоритму опорних векторів

Гістограма напрямлених градієнтів - алгоритм для розпізнання об'єктів на зображеннях за допомогою мета даних про інтенсивність і напрямки контурів об'єктів. Перед роботою дескриптора зображення підлягає попередній обробці: нормалізація кольорів, переведення у чорно-білі кольори, робота із контрастністю та шумом. Далі таке зображення розподіляється на сегменти і кожен піксель аналізується разом із сусідніми для визначення градієнту за допомогою порівняння показників яскравості: чим вище різниця між сусідніми пікселями, тим більший градієнт. На рисунку 2.4 зображено приклад визначення автомобіля за допомогою гістограми. [11]

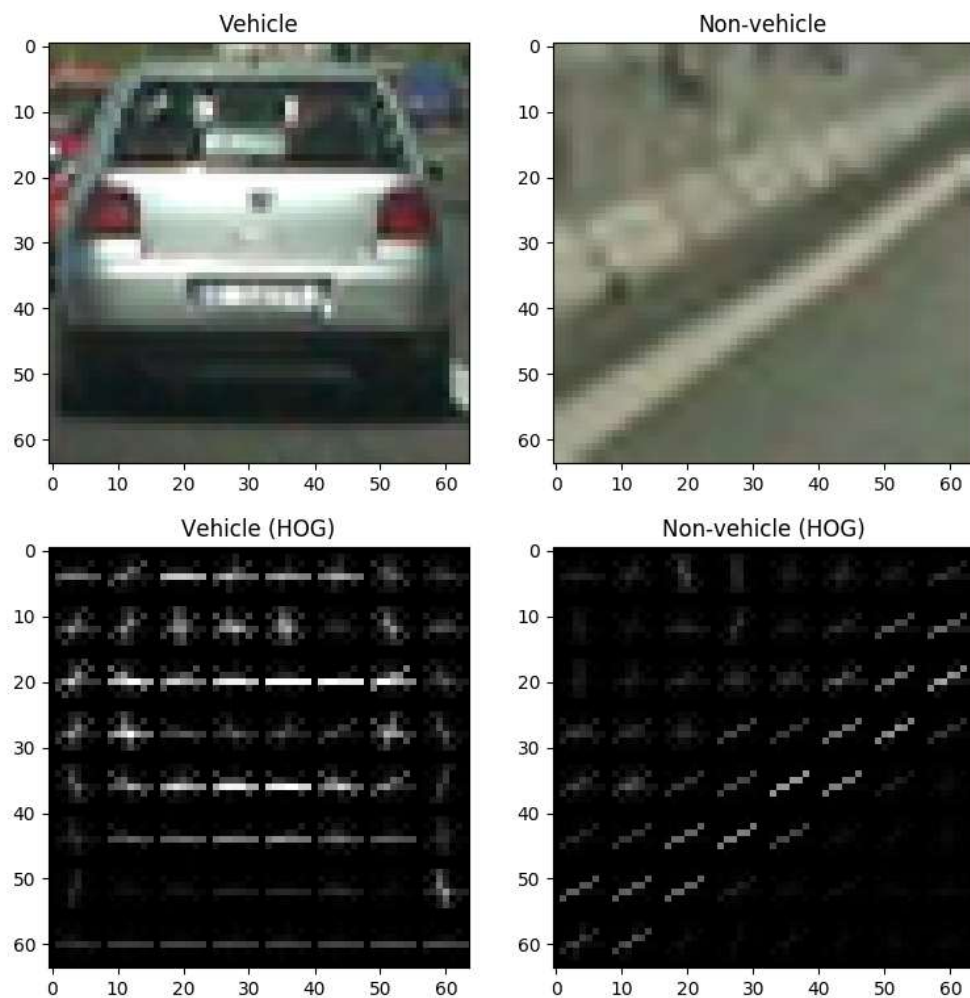


Рисунок 2.4 - Приклад роботи алгоритму

Поєднання цих алгоритмів в моделі дозволяє зменшити час на обробку зображення і визначення обличчя і його рис. Схема контрольних точок, яка використовується для розпізнавання рис обличчя, наведена на рисунку 2.5. [9]

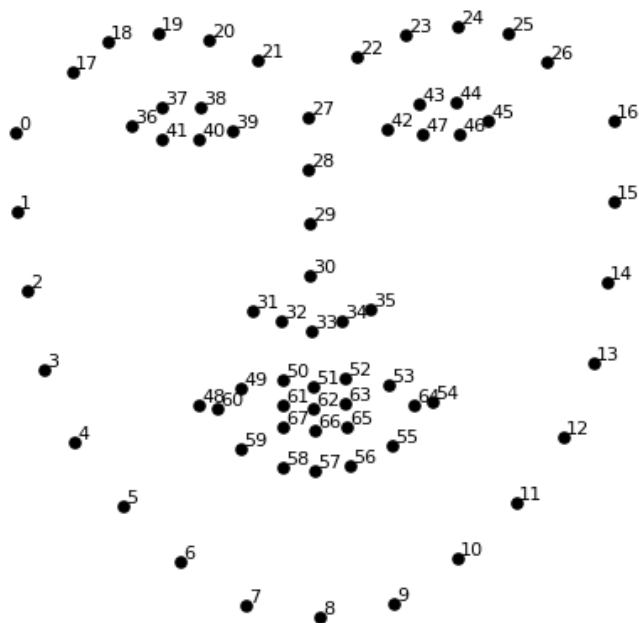


Рисунок 2.5 - Схема контрольних точок

За допомогою цієї схеми визначаються X та Y координати цих 68 точок на вхідному зображенні. На рисунку 2.6 зображено приклад роботи моделі на основі схеми контрольних точок із кількістю 194 вузлів.



Рисунок 2.6 - Приклад роботи моделі

Визначення основних частин обличчя дозволяє проаналізувати кожен із них на такі характеристики як загальний колір обличчя, почервоніння (кількість червоного кольору на зображенні), пігментацію (однobarвність обраного сегменту), темні кола під очима (контраст кольорів між щокою і областю під очима, порівняння по рівню сірого). Для визначення акне можна використовувати просту нейронну мережу, яка буде аналізувати уже обрані сегменти обличчя: лоб та щоки. Нейронна мережа буде згортковою, тобто один із кроків її алгоритму буде згортка, тобто створення нового зображення із вхідного за допомогою обрахування нового значення пікселів шляхом додавання сусідніх пікселів, попередньо зважених. Матриці згортки визначаються автоматично за допомогою навчання. Операція агрегування зменшує вхідний простір зображення, що допомагає боротися із проблемою зсувів і розташування об'єкту не в центрі зображення.[12]



Рисунок 2.7 - Принцип роботи згорткової нейронної мережі для класифікації об'єктів

Буде використовуватись попередньо натренована згорткова мережа ResNet-152, до якої будуть під'єднані декілька останніх шарів. Такий спосіб називається передавальне навчання, він дозволяє скоротити навчання мережі і зменшити розміри навчальної послідовності, оскільки більша кількість шарів уже попередньо навчена і містить малу кількість помилок, а заміна останніх шарів дозволяє налаштувати мережу для конкретної задачі. Мережа ResNet-152 складається із 152 шарів. Проблема таких великих мереж в тому, що глибокі шари важко навчити: стандартні методи із зворотнього поширення похибки до цих шарів не доходять, оскільки він стає занадто малим – ця проблема називається

проблемою зникнення градієнтів. Існує багато способів для вирішення цієї проблеми, такі як початкове заповнення вагових коефіцієнтів, додавання шарів нормалізації, попереднє навчання без вчителя тощо. [12]

До мережі було додано три шари з функцією активації у вигляді зрізаного лінійного вузлу (rectified linear unit, ReLU) (рисунок 2.8).

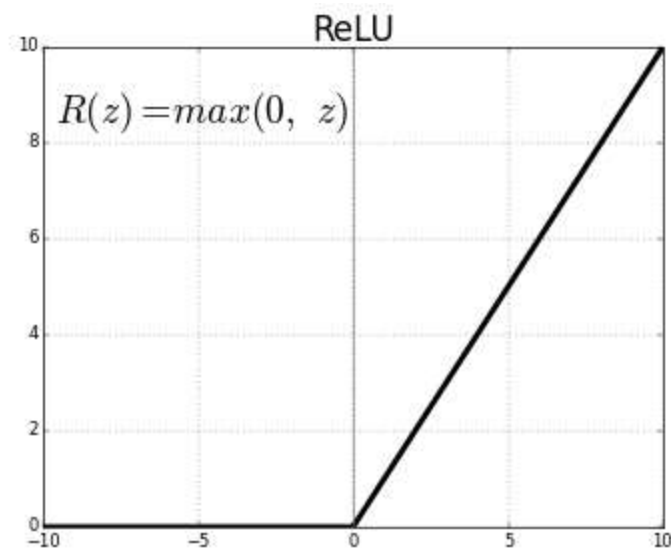


Рисунок 2.18 – ReLU функція

Вибір функції активації – один із важливих етапів розробки нейронної мережі, оскільки він визначає можливості функціоналу нейронної мережі і метод навчання. Оскільки у створеній мережі знаходиться лише три шари, то класичний алгоритм зворотного розповсюдження помилки працюватиме нормально. Цей алгоритм дозволяє навчати глибші шари нейронної мережі за допомогою обчислення похибки шляхом оптимізації вагових коефіцієнтів градієнтним спуском. Мережа була натренована на наборі даних, який складається із 989 зображень людей з акне різної тяжкості. У результаті натренована модель на валідаційних даних показує результат точності 99,51%.

2.5. Захист даних

Захист даних є важливою частиною розробки програмного забезпечення. Сучасні технології дозволяються розробникам задумуватись про захист даних

					КПІ.ІТ-8109.045440.02.81	Арк.
						39
Змін.	Арк.	№ докум.	Підп.	Дата.		

менше, ніж ще 20 років, коли сервіси надання хмарних послуг для розгортання проектів не були якісними і поширеними.

Безпосередньо у програмному забезпеченні більшість загроз вже уникнута засобами фреймворку Flask. Для запобігання несанкціонованого доступу до даних користувачів були використані токени при авторизації та автентифікації користувачів, а також функції для перевірки цих доступів. Також для захисту взаємодії між клієнтом та сервером використовується HTTPS з'єднання, налаштування HTTP-cookie. Окрім цього, сервіс для аналізу зображень являє собою AWS Lambda, захист даних якої покладено на сервіс Amazon, а конфігурація не дозволяє сторонніх запитів до функції, окрім застосунку.

Захист запитів до Lambda покладений на API Gateway, який фактично є маршрутизатором. Він реалізує усі задачі із обробкою запитів, керування трафіком, підтримку спільного використання ресурсів з різних джерел (Cross-Origin Resource Sharing, CORS). Принцип роботи зображено на рисунку 2.9.



Рисунок 2.9 – Принцип роботи API Gateway

					КПІ.IT-8109.045440.02.81	Арк.
						40
Змін.	Арк.	№ докум.	Підп.	Дата.		

2.6 Висновки до розділу

У даному розділі було зроблене моделювання та конструювання програмного забезпечення. Моделювання та аналіз представлено у вигляді BPMN-діаграми на прикладі двох складних бізнес-процесів. Була визначена тришарова архітектура програмного забезпечення, архітектура різних частин застосунку, взаємодія між ними. Також відповідно до вимог до проекту, архітектури та вигляду даних були обрані технології - мова програмування python і фреймворк flask, система управління реляційними базами даних PostgreSQL для сервісу керування користувачами і косметиними засобами, а також технологія AWSLambda і нереляційне сховище dynamodb для сервісу аналізу зображень. Була створена і описана даталогічна модель реляційної бази даних, були визначені основні пакети, модулі і класи застосунку, які зображено на діаграмах класів двох сервісів. Булі визначені і описані методи для роботи із зображеннями, для аналізу стану кольору шкіри людини - за допомогою моделі, яка використовує гістограму направлених градієнтів і метод опорних векторів і простого алгоритму для визначення основних особливостей, а також описано нейронну мережу, яка визначатиме стан акне.

					КПІ.IT-8109.045440.02.81	Арк.
						41
Змін.	Арк.	№ докум.	Підп.	Дата.		

3 АНАЛІЗ ЯКОСТІ ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Аналіз якості програмного забезпечення

Якість розроблюваного програмного забезпечення залежить від якості двох компонентів: якість моделі і іншої частини програмного забезпечення. Таблиця 3.1 показує результати тестування моделі при різній кількості набору даних.

Таблиця 3.1 – Результати тестування моделі.

Кількість даних	100	200	500	1000	2000
% помилок	.62	0.57	0.54	0.52	0.49

Якість іншої частини програмного забезпечення можна оцінити за допомогою тестування. Тестування повинно покривати усі функціональні вимоги.

3.2 Опис процесів тестування

Для проведення тестування необхідні перевірити усі функціональні вимоги. Тестування повинно проводитись не лише після закінчення реалізації продукту, а й під час його написання розробником. У програмному забезпеченні використовуються наступні рівні тестування:

- модульне тестування перевіряє функціональність найменших складових системи: методів, класів, функцій;
- інтеграційне тестування перевіряє взаємодію між компонентами системи;
- ручне тестування, яке перевіряє якість роботи кінцевого продукту.

Для тестування застосунку було обрано бібліотеку `pytets`, яка була розроблена для тестування проектів мовою `python`. Вона надає можливості для створення як модульних, так і інтеграційних тестів [13].

3.3 Опис контрольного прикладу

Модульні тести охоплюють маленькі сегменти коду і потребують великої кількості можливих варіантів використання функції, форматів даних і відповідей,

тому кількість модульних тестів на окрему функцію у окремих випадках досягає 15 штук. Це дозволяє зменшити кількість помилок на рівні інтеграційного тестування. Інтеграційні тести не потребують такої кількості тестів, тому їхня кількість набагато менша.

Далі у таблицях 3.1 - 3.8 наведені тести для перевірки основних функцій системи.

Таблиця 3.1 - Тест TC001

Мета тесту	Перевірка можливості входу в систему.
Початковий стан	Клієнт підключений до системи, відкрита сторінка логіну або запиту.
Вхідні дані	Дані користувача: пошта, пароль.
Схема проведення тесту	Користувач вводить свої дані у форму, відправляє їх до системи.
Очікуваний результат	Вхід виконано
Стан програмного продукту після проведення випробувань	Вхід виконано

Таблиця 3.2 - Тест TC002

Мета тесту	Перевірка можливості реєстрації користувача в системі.
Початковий стан	Клієнт підключений до системи, відкрита сторінка реєстрації або запиту.
Вхідні дані	Дані користувача: ім'я, прізвище, пошта, пароль.
Схема проведення тесту	Користувач вводить свої дані у форму, відправляє їх до системи.
Очікуваний результат	Користувача зареєстровано, вхід виконано.
Стан програмного продукту після проведення випробувань	Користувача зареєстровано, вхід виконано.

Таблиця 3.3 - Тест TC003

Мета тесту	Перевірка можливості завантаження зображення до системи.
Початковий стан	Відкрита сторінка для проведення аналізу шкіри.

Продовження таблиці 3.3

Вхідні дані	Зображення користувача.
Схема проведення тесту	Користувач натискає кнопку “завантажити зображення”, обирає зображення.
Очікуваний результат	Зображення завантажено до системи.
Стан програмного продукту після проведення випробувань	Зображення завантажено до системи.

Таблиця 3.4 - Тест TC004

Мета тесту	Перевірка можливості проведення аналізу шкіри.
Початковий стан	Клієнт підключений до системи, відкрита сторінка проведення аналізу, зображення завантажено.
Вхідні дані	Дані користувача: зображення.
Схема проведення тесту	Користувач натискає кнопку “провести результат”.
Очікуваний результат	Результат аналізу шкіри по пунктам, надано перелік рекомендованих косметичних засобів.
Стан програмного продукту після проведення випробувань	Результат аналізу шкіри по пунктам, надано перелік рекомендованих косметичних засобів.

Таблиця 3.5 - Тест TC005

Мета тесту	Перевірка можливості відправити результат аналізу на пошту.
Початковий стан	Клієнт підключений до системи, аналіз було проведено. Користувач не ввійшов в систему.
Вхідні дані	Дані користувача: пошта.
Схема проведення тесту	Користувач натискає кнопку “відправити на пошту”, вводить дані пошти у форму, натискає кнопку “відправити”.
Очікуваний результат	Результат аналізу відправлено на пошту користувача.
Стан програмного продукту після проведення випробувань	Результат аналізу відправлено на пошту користувача.

Таблиця 3.6 - Тест TC006

Мета тесту	Перевірка можливості відправити результат аналізу на пошту зареєстрованого користувача.
Початковий стан	Клієнт підключений до системи, аналіз було проведено. Користувач увійшов в систему.
Вхідні дані	Відсутні.
Схема проведення тесту	Користувач натискає кнопку “відправити на пошту”.
Очікуваний результат	Результат аналізу відправлено на пошту користувача.
Стан програмного продукту після проведення випробувань	Результат аналізу відправлено на пошту користувача.

Таблиця 3.7 - Тест TC007

Мета тесту	Перевірка можливості перегляду минулих результатів аналізу.
Початковий стан	Клієнт авторизований у системи, аналіз було проведено. Користувач на сторінці перегляду профілю.
Вхідні дані	Відсутні.
Схема проведення тесту	Користувач натискає кнопку “минулі результати”.
Очікуваний результат	Результат аналізу і рекомендації косметичних засобів відображені.
Стан програмного продукту після проведення випробувань	Результат аналізу і рекомендації косметичних засобів відображені.

Таблиця 3.8 - Тест TC008

Мета тесту	Перевірка можливості зміни даних користувача
Початковий стан	Клієнт авторизований у системи. Клієнт на сторінці свого профілю.
Вхідні дані	Дані, які користувач хоче змінити: ім'я, прізвище, пароль, пошта.
Схема проведення тесту	Користувач натискає кнопку «редагувати профіль», вводить дані, які хоче змінити, натискає кнопку «змінити»

Продовження таблиці 3.8

Очікуваний результат	Дані користувача змінені. У випадку зміни пошти – надсилання листа для підтвердження нової пошти.
Стан програмного продукту після проведення випробувань	Дані користувача змінені. У випадку зміни пошти – надсилання листа для підтвердження нової пошти.

3.4 Висновок до розділу

У даному розділі була проаналізована якість програмного забезпечення. Оскільки програмне забезпечення можна поділити на дві складові: математичну (алгоритм аналізу зображення) і програмну складову (API), то оцінка якості була у визначенні кількості помилок роботи алгоритму і модульному, інтеграційному та ручному тестуванні відповідно. У результаті тестування алгоритму на даних для валідації була визначена його точність - 99,51%.

Для тестування програмної складової були написані модульні та інтеграційні тести. Основні тест-плани кінцевого програмного забезпечення були наведені у таблицях 3.1-3.8. Результатом тестування є позитивний результат за всіма кейсами, що означає, що розроблене програмне забезпечення задовольняє поставлені функціональні вимоги.

4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Розгортання розгортання програмного забезпечення

Програмне забезпечення являє собою веб-застосунок у вигляді двох сервісів-API. Сервіси можуть бути використаними у вигляді backend частини застосунку або як самостійний проект API лише для аналізу зображень, для чого необхідно змінити налаштування AWS Lambda, а саме доступи та білий список клієнтів. Для розгортання проекту необхідно мати підключення до інтернету, а також аккаунт AmazonWebServices, налаштований для роботи.

Розгортання відбувається за наступною інструкцією:

- зробити деплой та запуск AWS Lambda;
- перевірити доступність портів;
- розгорнути бази даних на віддаленому сервері;
- розгорнути застосунок на віддаленому сервері.

4.2 Робота із програмним забезпеченням

Робота із програмним забезпеченням детально описана у керівництві користувача.

4.3 Висновки до розділу

У розділі була описана послідовність дій для розгортання програмного забезпечення на віддаленому сервері. Завдяки можливостям використаних технологій, такі як AmazonWebServices і docker-compose розгортання є досить простим, але конфігурація і вирішення помилок даних сервісів може вимагати додаткових знань.

					КПІ.IT-8109.045440.02.81	Арк.
						47
Змін.	Арк.	№ докум.	Підп.	Дата.		

ВИСНОВКИ

Під час виконання дипломного проекту було розроблено програмне забезпечення для розпізнавання властивостей та проблем шкіри засобами штучного інтелекту. Результат розпізнавання використовується для аналізу стану шкіри і створенню рекомендацій косметичних засобів.

Першим кроком розробки програмного забезпечення є аналіз предметної області, відомих технічних рішень та проектів. Завдяки цьому можна визначити переваги і недоліки існуючого програмного забезпечення, визначити основні функціональні та нефункціональні вимоги, продумати варіанти використання програмного забезпечення.

Наступним кроком є моделювання і конструювання програмного забезпечення. Після визначення із вимогами до проекту, було описано основні бізнес процеси, які дозволили визначити складові системи, задачі для проектування, кількість та вигляд даних для роботи. Завдяки цьому була обрана необхідні архітектура та технології, створені схеми та діаграми, за якими було реалізовано програмний продукт. Додатково для демонстрації роботи продукту було створено клієнтський застосунок.

Останнім кроком перед впровадженням програмного забезпечення була оцінка якості продукту. Були проведені різні способи тестування на всіх етапах розробки: як безпосередньо під час реалізації проекту у вигляді модульних та інтеграційних тестів, так і тестування готового продукту. Основні тести визначені у контрольному прикладі. За результатами тестування можна зробити висновок, що програмне забезпечення виконує усі функціональні вимоги, а точність системи аналізу зображень є задовільною - 99,51%.

Останнім пунктом у розробці програмного забезпечення є впровадження і розгортання. Процес визначено у відповідному розділі, а опис проекту для користувачів детально описано у керівництві користувача.

					КПІ.IT-8109.045440.02.81	Арк.
						48
Змін.	Арк.	№ докум.	Підп.	Дата.		

ВИКОРИСТАНІ ДЖЕРЕЛА

1. Matthias Biehl. RESTful API Design: Best Practices in API Design with REST - API-University Press; 1st edition, 2016 – 274 с.
2. Stefan Duffner. Face Image Analysis With Convolutional Neural Networks, 2007.
3. C. Garcia and M. Delakis. Convolutional face finder: A neural architecture for fast and robust face detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2004
4. Peter Bruce, Andrew Bruce. Practical Statistics for Data Scientists - O'Reilly Media, Inc., 2017 – 425 с.
5. Shang-Hung Lin, S. Kung, Long-Ji Lin. Face recognition/detection by probabilistic decision-based neural network, 1997
6. Miguel Grinberg, Flask Web Development: Developing Web Applications with Python - O'Reilly Media, 2nd Edition, 2018 – 310 с.
7. AWS Documentation [Електронний ресурс] / Amazon - Режим доступу: https://docs.aws.amazon.com/index.html?nc2=h_q1_doc_do (06.06.2022)
8. Andreas Meier and Michael Kaufmann. SQL & NoSQL Databases: Models, Languages, Consistency Options and Architectures for Big Data Management, 2019 – 229 с.
9. Vahid Kazemi, Josephine Sullivan. One Millisecond Face Alignment with an Ensemble of Regression Trees, 2014
10. Dmitriy Fradkin, Ilya Muchnik. Support vector machines for classification, 2006.
11. Navneet Dalal, Bill Triggs. Histograms of Oriented Gradients for Human Detection, 2005
12. Ian Goodfellow, Yoshua Bengio, Aaron Courville. Deep Learning (Adaptive Computation and Machine Learning series) - The MIT Press; Illustrated edition, 2016 – 800 с.
13. Brian Okken. Python Testing with pytest: Simple, Rapid, Effective, and Scalable - Pragmatic Bookshelf, 2017 - 222 с.

					КПІ.IT-8109.045440.02.81	Арк.
						49
Змін.	Арк.	№ докум.	Підп.	Дата.		

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ РОЗПІЗНАВАННЯ
ВЛАСТИВОСТЕЙ ТА ПРОБЛЕМ ШКІР ЛЮДИНИ ЗАСОБАМИ
ШТУЧНОГО ІНТЕЛЕКТУ**

Опис програми

КПІ.IT-8109.045440.03.13

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Діна ІОВЧЕВА

Київ – 2022

ТЕКСТ ПРОГРАМНОГО КОДУ

Тексти програмного коду

Програмне забезпечення для розпізнавання властивостей та проблем шкір людини засобами штучного інтелекту

(Найменування програми (документа))

CD-R

(Вид носія даних)

18 арк, 514700 Кб

(Обсяг програми, арк., Кб)

Київ – 2022

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

dermalab/dermalab/core.py

```
db = SQLAlchemy() #: Flask-SQLAlchemy extension instance
```

```
mail = Mail() #: Flask-Mail extension instance
```

```
login_manager = LoginManager() #: Flask-Login extension
```

```
babel = Babel() #: flask-babel
```

```
cors = CORS() # cross origin support (for the API)
```

```
cache = Cache()
```

```
mm = Marshmallow()
```

```
class BeautilabError(Exception):
```

```
    def __init__(self, msg, code=500): # pylint: disable=super-init-not-called
```

```
        self.msg = msg
```

```
        self.code = code
```

```
class Service:
```

```
    __model__ = None
```

```
    def __init__(self, service_registry):
```

```
        self.services = service_registry
```

```
    @property
```

```
    def session(self):
```

```
        return db.session
```

```
    @staticmethod
```

```
    def query_result_to_dict(result):
```

```
        return dict(zip(result.keys(), list(result)))
```

```
    @staticmethod
```

```
    def expunge(model):
```

```
        db.session.expunge(model)
```

```
    def get_model(self):
```

```
        return self.__model__
```

```
    @staticmethod
```

```
    def call_procedure(function_name, params):
```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

```

connection = db.engine.raw_connection()
try:
    cursor = connection.cursor()
    cursor.callproc(function_name, params)
    results = list(cursor.fetchall())
    cursor.close()
    connection.commit()
    db.session.commit()
    return results
finally:
    connection.close()

def get_id_by_uuid(self, uuid):
    return self.__model__.query.with_entities(self.__model__.id).filter(self.__model__.uuid ==
uuid).scalar()

def get_uuid_by_id(self, id: int):
    return self.__model__.query.with_entities(self.__model__.uuid).filter(self.__model__.id ==
id).scalar()

@staticmethod
def commit():
    db.session.commit()

@staticmethod
def refresh(entity):
    db.session.refresh(entity)

@staticmethod
def exists_with_values(model, kwargs):
    return model.query.filter_by(**kwargs).count() > 0

@staticmethod
def attach_to_session(instance):
    state = inspect(instance)
    if state.detached:
        db.session.add(instance)

```

```

@staticmethod
def add_list_to_session(*instances):
    db.session.add_all(instances)

def _isinstance(self, model, raise_error=True):
    is_instance = isinstance(model, self.__model__) # pylint: disable=isinstance-second-argument-not-
valid-type
    if not is_instance and raise_error:
        raise ValueError('%s is not of type %s' % (model, self.__model__))
    return is_instance

def save(self, model):
    self._isinstance(model)
    db.session.add(model)
    db.session.commit()
    db.session.refresh(model)

    return model

def delete_model_list(self, models):
    for model in models:
        self._isinstance(model)
        db.session.delete(model)
        db.session.commit()

def all(self):
    return self.__model__.query.all()

def get(self, id):
    return self.__model__.query.get(id)

def get_by_uuid(self, uuid):
    try:
        return self.__model__.query.filter_by(uuid=uuid).one()
    except exc.SQLAlchemyError:
        abort(404)
    return None

```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

```

def get_or_create(self, **kwargs):
    try:
        instance = self.__model__.query.filter_by(**kwargs).one()
        return instance
    except exc.SQLAlchemyError:
        return self.create(**kwargs)

def get_or_create_model(self, **kwargs):
    try:
        instance = self.__model__.query.filter_by(**kwargs).one()
        return instance
    except exc.SQLAlchemyError:
        return self.new_model(**kwargs)

def select(self, query, column_names):
    columns = []

    for name in column_names:
        columns.append(getattr(self.__model__, name))

    return query.with_entities(*columns)

def get_all_by_uuid(self, *uuids):
    if hasattr(self.__model__, 'uuid'):
        return self.__model__.query.filter(self.__model__.uuid.in_(uuids)).all()
    raise AttributeError("{} has no uuid column".format(self.__model__.__class__.__name__))

def get_all_by_ids(self, ids):
    if hasattr(self.__model__, 'id'):
        return self.__model__.query.filter(self.__model__.id.in_(ids)).all()
    raise AttributeError("{} has no id column".format(self.__model__.__class__.__name__))

def find(self, **kwargs):
    return self.__model__.query.filter_by(**kwargs)

def get_filtered_list(self, **kwargs):
    return self.find(**kwargs).all()

```



```

def get_count(self, **kwargs):
    return self.find(**kwargs).count()

def first(self, **kwargs):
    return self.find(**kwargs).first()

def get_or_404(self, id):
    return self.__model__.query.get_or_404(id)

def new(self, **kwargs):
    return self.__model__(**self._preprocess_params(kwargs)) # pylint: disable=not-callable

def create(self, **kwargs):
    return self.save(self.new(**kwargs))

def update(self, model, **kwargs):
    self._isinstance(model)
    model = self.update_model(model, **kwargs)
    self.save(model)

    return model

def delete(self, model):
    self._isinstance(model)
    db.session.delete(model)
    db.session.commit()

def get_max_id(self):
    return db.session.query(db.func.max(self.__model__.id)).scalar()

def get_column(self, name, mapped_fields, allow_many=False):
    column = mapped_fields.get(name, None)
    if column is None:
        column = getattr(self.__model__, name, None)

    if isinstance(column, list):
        return column[0] if not allow_many else column

```

```
return column
```

```
@staticmethod
```

```
def mark_deleted(instance):
```

```
    db.session.delete(instance)
```

```
@classmethod
```

```
def mark_list_deleted(cls, instances):
```

```
    for instance in instances:
```

```
        cls.mark_deleted(instance)
```

```
def load_relationships(
```

```
    self,
```

```
    query,
```

```
    lazy: Iterable = (),
```

```
    joined: Iterable = (),
```

```
    subquery: Iterable = (),
```

```
    selectin: Iterable = (),
```

```
    raised: Iterable = (),
```

```
    noload_: Iterable = (),
```

```
    noload_other: bool = False
```

```
):
```

```
    load_options: list = []
```

```
for field in inspect(self.__model__).relationships.keys():
```

```
    if field in lazy:
```

```
        load_options.append(lazyload(field))
```

```
    elif field in joined:
```

```
        load_options.append(joinedload(field))
```

```
    elif field in subquery:
```

```
        load_options.append(subqueryload(field))
```

```
    elif field in selectin:
```

```
        load_options.append(selectinload(field))
```

```
    elif field in raised:
```

```
        load_options.append(raiseload(field))
```

```
    elif field in noload_ or noload_other:
```

```
        load_options.append(noload(field))
```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		8

```

return query.options(*load_options)

def get_by_uuid_with_relationships(
    self,
    uuid: str,
    relationships_options: Dict[str, Iterable] = None
) -> Union[__model__, None]:
    query: 'Query' = self.__model__.query.filter_by(
        uuid=uuid
    )

    return self.load_relationships(query, **(relationships_options or {})).first()

def select_only(
    self,
    filters_by: Union[dict, list],
    columns: list,
    first: bool = True,
    is_complex: bool = False,
    options: Optional[list] = None
):
    query = db.session.query(
        *[getattr(self.__model__, column) for column in columns]
    ).select_from(
        self.__model__
    )

    if options:
        query = query.options(
            load_only(*columns),
            *options
        )

    if filters_by:
        if is_complex:
            query = query.filter(*filters_by)
        else:
            query = query.filter_by(**filters_by)

```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		9

```

    if first:
        return query.first()

    return query.all()

def load_only(
    self, filters_by: Union[dict, list], columns: list, options: list, first: bool = True,
    order_by: Optional[list] = None
):
    query = db.session.query(
        self.__model__
    ).options(
        load_only(*columns),
        *options
    )

    if isinstance(filters_by, list):
        query = query.filter(*filters_by)
    else:
        query = query.filter_by(**filters_by)

    if order_by:
        query = query.order_by(*order_by)

    if first:
        return query.first()

    return query.all()

```

dermalab/dermalab/resources/analysis/models.py

```

analysis_products_map = db.Table(
    'analysis_products_map',
    db.metadata,
    db.Column(
        'product_id',

```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		10

```

        db.Integer(),
        db.ForeignKey('product.id', ondelete='CASCADE'),
        nullable=False
    ),
    db.Column(
        'analysis_id',
        db.Integer(),
        db.ForeignKey('analysis.id', ondelete='CASCADE'),
        nullable=False
    )
)

class Analysis(db.Model, TimeStampMixin):
    __tablename__ = 'analysis'

    id = db.Column(db.Integer(), primary_key=True)
    uuid = db.Column(db.String(36), default=uuid.uuid4, nullable=False, unique=True)
    category_type = db.Column(CategoryType.db_type(), nullable=False)
    user_id = db.Column(db.Integer(), nullable=False)
    picture_url = db.Column(db.String(255), nullable=False)

    user = db.relationship('User', back_populates='analysis')

    products = db.relationship(
        'Products',
        single_parent=True,
        secondary=analysis_products_map,
        cascade='all',
        passive_deletes=True
    )

```

dermalab/dermalab/resources/user/models.py

```

class User(db.Model, TimeStampMixin):
    __tablename__ = 'user'

    id = db.Column(db.Integer(), primary_key=True)
    uuid = db.Column(db.String(36), default=uuid.uuid4, nullable=False, unique=True)
    user_email = db.Column(db.String(255), unique=True, nullable=False)

```

					КПІ.IT-8109.045440.03.13	Арк.
						11
Змін.	Арк.	№ докум.	Підп.	Дата.		

```

user_firstname = db.Column(db.String(120))
user_lastname = db.Column(db.String(120))
user_password = db.Column(db.String(130))

username = column_property(user_firstname + ' ' + user_lastname + ' ' + user_firstname)

analysis = db.relationship(
    'Analysis',
    back_populates='user',
    cascade='all, delete',
    passive_deletes=True
)

@hybrid_property
def full_name(self):
    """
    Full name getter
    :return:
    """
    return '{0} {1}'.format(self.user_firstname, self.user_lastname)

@full_name.expression
def full_name(self):
    """
    Get User full name
    :return: string
    """
    return db.func.trim(func.concat(self.user_firstname, ' ', self.user_lastname))

```

dermalab/dermalab/resources/products/models.py

```

product_functionality_map = db.Table(
    'product_functionality_map',
    db.metadata,
    db.Column(
        'product_id',
        db.Integer(),
        db.ForeignKey('product.id', ondelete='CASCADE'),

```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		12

```

        nullable=False
    ),
    db.Column(
        'functionality_id',
        db.Integer(),
        db.ForeignKey('functionality.id', ondelete='CASCADE'),
        nullable=False
    )
)

class Functionality(db.Model, TimeStampMixin):
    __tablename__ = 'functionality'

    id = db.Column(db.Integer(), primary_key=True)
    uuid = db.Column(db.String(36), default=uuid.uuid4, nullable=False, unique=True)

    title = db.Column(db.String(120), unique=True, nullable=False)

class Category(db.Model):
    __tablename__ = 'category'

    id = db.Column(db.Integer(), primary_key=True)
    uuid = db.Column(db.String(36), default=uuid.uuid4, nullable=False, unique=True)
    title = db.Column(db.String(120), unique=True, nullable=False)
    category_type = db.Column(CategoryType.db_type())

    products = db.relationship(
        'Product',
        back_populates='category',
        cascade='all, delete',
        passive_deletes=True
    )

class Product(db.Model, TimeStampMixin):
    __tablename__ = 'product'

```

```

id = db.Column(db.Integer(), primary_key=True)
uuid = db.Column(db.String(36), default=uuid.uuid4, nullable=False, unique=True)
product_name = db.Column(db.String(120), unique=True, nullable=False)

description = db.Column(db.Text(), nullable=True)
size = db.Column(db.Integer(), nullable=False)
ingredients = db.Column(db.Text(), nullable=False)

picture = db.Column(db.String(255), nullable=True)

category_id = db.Column(
    db.Integer(),
    db.ForeignKey('category.id', ondelete='CASCADE'),
    nullable=False
)

category = db.relationship('Category', back_populates='products')
functions = db.relationship(
    'Functionality',
    single_parent=True,
    secondary=product_functionality_map,
    cascade='all',
    passive_deletes=True
)

```

dermalab/dermalab/api/helpers/auth.py

```
REMEMBER_ME_COOKIE_KEY = 'user_login'
```

```
def login():
```

```
    form = UserLoginForm()
```

```
    if current_user.is_authenticated and current_user.user_email != form.email.data:
```

```
        logout()
```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		14


```

if form.validate():
    try:
        user = UserAuthHelper.authorise_user(form.email.data, form.password.data)
        process_login(user, form)
        return Response(user=session.get('user')).dump()

    except BeutilabError as error:
        response = Response(success=False, errors=[error.msg], user=None).dump()

    return response

def process_login(user, login_form):
    session.permanent = True

    usr = {
        'user_firstname': user.user_firstname,
        'user_lastname': user.user_lastname,
        'user_email': user.user_email,
        'user_uuid': user.uuid,
    }

    session['user'] = usr
    login_user(user=user, force=True)

    response = make_response()

    expire_days = 90 if login_form.remember_me.data else -1
    expire_date = datetime.now() + timedelta(days=expire_days)
    response.set_cookie(REMEMBER_ME_COOKIE_KEY, user.user_email, expires=expire_date)

    return response

def logout():
    session.pop('user', None)

    logout_user()

```

```

def create_account():
    if current_user.is_authenticated:
        return Response(user=session.get('user')).dump()

    form = UserRegisterForm()

    if form.validate():
        user_data = build_new_user(form)
        user = services.users.first(user_email=user_data['user_email'])

        if user:
            return Response(success=False, errors=['Email or Password is incorrect'])

        services.users.create_user(user_data)

    return Response(user=session.get('user')).dump()

```

dermalab/dermalab/resources/helpers/s3_path_manager.py

```

class S3PathManager:

    image_path = '/analysis/{0}'
    user_images_path = '/users/{0}/analysis/{1}'
    AWS_HOST = 'https://{0}.s3.amazonaws.com{1}'

    def __init__(self, services=None, *args, **kwargs):
        self.services = services

    def build_responses_path(self, user_id, analysis_id):
        return self.user_images_path.format(user_id, analysis_id) if user_id \
            else self.image_path.format(analysis_id)

    @classmethod
    def s3_cdn_path(cls, path):
        return cls.AWS_HOST.format(current_app.config['CDN_BUCKET'], path)

```

dermalab/dermalab/api/utils.py

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		16

```

def upload_file(file, event_uuid, file_prefix=None) :
    path_manager = S3PathManager(services)
    file_uuid = uuid.uuid4()
    filename = file.filename

    if file_prefix:
        filename = '{}/{{}'.format(file_prefix, filename)

    bucket_name = '{0}/{1}'.format(file_uuid, filename)

    key_s3 = path_manager.build_responses_path(event_uuid, bucket_name)
    file_name = path_manager.s3_cdn_path(key_s3)

    if file:
        file_content_type = file.content_type
        temp_file = create_temp_file(delete=False)
        file.save(temp_file)
        temp_file.flush()
        s3_put_file_asset.apply_async(args=[key_s3, temp_file.name, file_content_type])

    return file_uuid, file_name

```

dermalab/dermalab/api/pictures.py

```

@route(events_bp, '/<event_uuid>/nomination/media', methods=['POST'])
@csrf.exempt
def upload_media(event_uuid):
    file_prefix = None
    if context_uuid:
        file_prefix = S3PathManager.build_media_file_prefix(context_uuid, 'form')

    bucket_uuid, file_link = upload_file(next(request.files.values()), event_uuid, file_prefix)
    return {'data': file_link}

```

dermalab/dermalab/api/analysis.py

```

@route(events_bp, '/<event_uuid>/nomination/media', methods=['POST'])

```

					КПІ.IT-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		17

```

@csrf.exempt
def create_analysis():
    form = request.form()
    return services.analysis.create_analysis(session['user_id'], form.image_url.data)

```

dermalab/dermalab/resources/aws_dynamo.py

```

class AWSDynamoService(Service):
    @DynamoClients.set_dynamo_clients
    def get_digital_event_dynamo_table(self, event_uuid: str): # pylint: disable=no-self-use
        """
        Get DigitalEvent Table in AWS DynamoDB
        :param event_uuid: Event.uuid
        :return: DigitalEvent instance or None
        """
        table_name = DigitalEvent.table_name_tpl.format(stage=current_app.config['DYNAMO_STAGE'])

        if DigitalEvent.is_table_exist(table_name):
            return DigitalEvent(stage=current_app.config['DYNAMO_STAGE'], event_uuid=event_uuid)

        return None

    @DynamoClients.set_dynamo_clients
    def create_dynamo_table(self, event: 'Event'):
        dermalab_table = None

        try:
            dermalab_table = DermalabTable(
                stage=current_app.config['DYNAMO_STAGE'],
                tags=dict(
                    ENV=current_app.config['DYNAMO_STAGE'],
                    CreatedAt=DigitalEvent.get_now()
                )
            )
        except Exception as error: # pylint: disable=broad-except
            print(f'[DynamoDB ERROR]: {error}')

        return dermalab_table

```

dermalab/dermalab/resources/analysis.py

```
def create_analysis(self, file_url, user_id):
    AWS_HOST = 'https://{0}.s3.amazonaws.com{1}'

    if file_url and user_id:
        request = flask.form_request(json={file_url=file_url, user_id=user_id})

    response = requests.post(request)
    analysis_id = response.get('analysis_id', None)

    return services.aws_dynamo.get_analysis(analysis_id)
```

dermalab/aws/serverless/dermalab/http_api/api.py

```
@app.route('/create_analysis', methods=['POST'])
@load_authorizer_data
def create_analysis(user_info) -> dict:
    file_url = requests.json.get('file_url', None)
    user_id = user_info.get('user_id', None)
    result = None
    if file_url:
        result = dermalab.create_analysis(file_url, user_id)
        response = make_response(Response(result=result).dump())
    else:
        response = make_fail_response(FailResponse(msg='File url is not correct').dump())

    response.headers['Access-Control-Allow-Origin'] = '*'
    return response
```

dermalab/aws/layers/dermalab/python/models/mixins/analysis.py

```
from ...enums import Levels
```

```

class AnalysisMixin:
    def create_analysis(file_url, user_id) -> List[dict]:
        image = preprocessing.image.load_img(image_path)
        image_array = preprocessing.image.img_to_array(image)
        scaled_img = np.expand_dims(image_array, axis=0)
        model = models.load_model('Acne_grade.h5')
        pred = model.predict(scaled_img)
        output_acne_level = Levels.values[np.argmax(pred.acne.level)]

        return dict(result=dict(redness=pred.result.redness, pigmentation=pred.result.pigmentstion,
        acne=output_acne_level, data=pred))

```

					КПІ.ІТ-8109.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		20

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ РОЗПІЗНАВАННЯ
ВЛАСТИВОСТЕЙ ТА ПРОБЛЕМ ШКІРИ ЛЮДИНИ ЗАСОБАМИ
ШТУЧНОГО ІНТЕЛЕКТУ**

Програма та методика тестування

КПІ.IT-8109.045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Діна ІОВЧЕВА

Київ – 2022

ЗМІСТ

1 Об'єкт випробувань	3
2 Мета тестування	4
3 Методи тестування.....	5
4 Засоби та порядок тестування.....	6

					КПІ.ІТ-8109.045440.04.51	Арк.
						2
Змін.	Арк.	№ докум.	Підп.	Дата.		

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту “DermaLab”.

					КПІ.ІТ-8109.045440.04.51	Арк.
						3
Змін.	Арк.	№ докум.	Підп.	Дата.		

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення відповідно до функціональних вимог;
- знаходження проблем та недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

					КПІ.IT-8109.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи: статичне тестування – перевіряється вся документація, яка аналізується на предмет дотримання стандартів програмування;

- модульне тестування - перевіряється функціональність найменших складових системи: методів, класів, функцій;
- інтеграційне тестування - перевіряється взаємодія між компонентами системи;
- ручне тестування - перевірка роботи продукту.

					КПІ.IT-8109.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується вручну, програмними засобами та засобами, написаними реалізованими у процесі реалізації програмного забезпечення.

Тестування програмними засобами проводиться безпосередньо під час написання програмного коду із використанням таких засобів як середовище розробки, а також бібліотеки Python flake8. За допомогою цих програмних засобів виконується тестування коду на відповідність стандарту програмування мовою Python - PEP8.

Модульне тестування використовується для перевірки коректності роботи окремих компонентів. Для цього було обрано бібліотеку pytest. За допомогою модульних тестів перевіряється робота маленьких сегментів коду, тому їхня кількість досить велика. За допомогою утиліти coverage можна дізнатись якість покриття коду, зображену у відсотках протестованих варіантів використання функції, класу або методу. За результатами перевірки програмне забезпечення протестовано на 83.5%.

Інтеграційне тестування використовується для тестування взаємодії модулів, компонентів, складових системи. Для цього було обрано бібліотеку pytest.

Ручне тестування, з метою знаходження помилок та недоліків як у функціональній частині програмного забезпечення так і в зручності в користуванні. Для того, щоб перевірити працездатність та відмовостійкість додатку, необхідно провести наступні тестування:

- перевірку можливості входу в систему;
- перевірку можливості реєстрації користувача в системі;
- перевірку можливості виходу із системи;
- перевірку можливості завантаження зображення до системи;
- перевірку можливості проведення аналізу;
- перевірку можливості відправлення результату на пошту незареєстрованого користувача;

					КПІ.IT-8109.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

- перевірку можливості відправити результат на пошту зареєстрованого користувача;
- перевірка можливості перегляду минулих результатів аналізу користувача;
- перевірка можливості зміни даних користувача.

					КПІ.ІТ-8109.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ РОЗПІЗНАВАННЯ
ВЛАСТИВОСТЕЙ ТА ПРОБЛЕМ ШКІРИ ЛЮДИНИ ЗАСОБАМИ
ШТУЧНОГО ІНТЕЛЕКТУ**

Керівництво користувача

КПІ.IT-8109.045440.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Діна ІОВЧЕВА

Київ – 2022

ЗМІСТ

1 Загальні відомості	3
2 Підготовка до роботи.....	4
2.1 Системні вимоги для коректної роботи	4
2.2 Завантаження застосунку.....	4
2.3 Перевірка коректної роботи.....	4
3 Робота із застосунком.....	5

					КПІ.ІТ-8109.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

1 ЗАГАЛЬНІ ВІДОМОСТІ

«DermaLab» - це вебзастосунок для автоматизації аналізу шкіри і створення рекомендацій косметичних засобів.

					КПІ.IT-8109.045440.05.34	Арк.
						3
Змін.	Арк.	№ докум.	Підп.	Дата.		

2 ПІДГОТОВКА ДО РОБОТИ

2.1 Системні вимоги для коректної роботи

Для успішної роботи даного застосунку необхідне виконання наступних вимог:

- доступ до інтернету;
- веббраузер актуальних версій пізніше 2015.

2.2 Завантаження застосунку

Застосунок не потрібно завантажувати, оскільки він працює у веббраузері.

2.3 Перевірка коректної роботи

Перевірка коректності роботи полягає у коректному завантаженні головного вікна вебсайту. (Рисунок 3.1).

					КПІ.IT-8109.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 РОБОТА ІЗ ЗАСТОСУНКОМ

При відкритті вебсайту буде завантажена головна сторінка (рисунки 3.1). На ній вказана інформація про вебсайт (бренд косметики) і є панель навігації для переходу по сторінкам. Наповненням сторінки слугує перелік косметичних засобів.

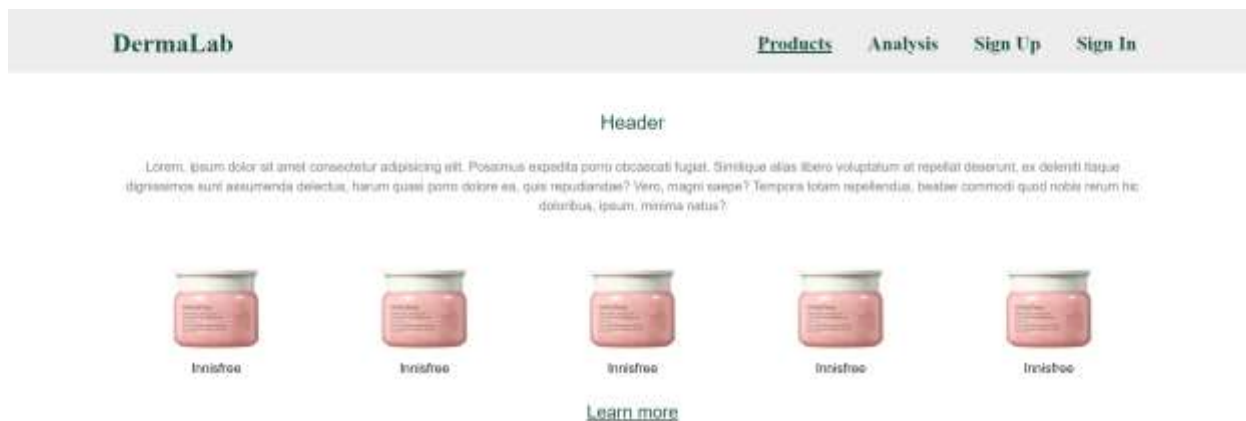


Рисунок 3.1 – Початкова сторінка додатку

З сторінки можна перейти на сторінку логіну чи реєстрації (рисунки 3.2 і 3.3).



Рисунок 3.2 - Сторінка логіну

Рисунок 3.3 - Сторінка реєстрації

Також користувач має можливість перейти до сторінки проведення аналізу. (Рисунок 3.4). Початкова сторінка аналізу містить у собі поле для завантаження фотографії.

Рисунок 3.4

Після завантаження фотографії користувач має доступною кнопку “зробити аналіз” і може провести аналіз (рисунок 3.5).



Рисунок 3.5 - Завантажене зображення

Після проведення аналізу користувачеві зображується результат аналізу: стан шкіри, рекомендовані косметичні засоби по категоріям - очищення, зволоження, догляд, захист від сонця. (Рисунки 3.6)

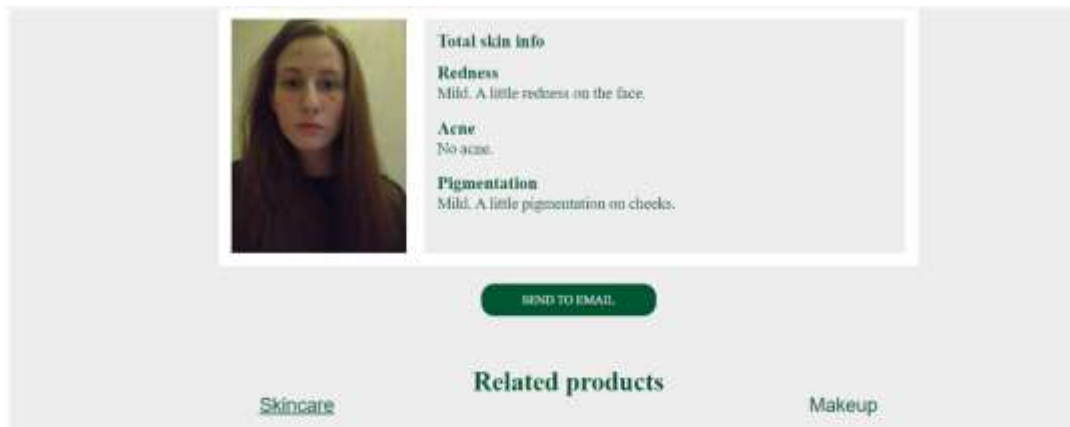


Рисунок 3.6 - Результат аналізу

При натисканні кнопки “відправити на пошту” користувачеві відкривається модальне вікно, куди йому треба ввести пошту, якщо він не авторизований. Якщо користувач у системі, то вікно не з’явиться і результат відправиться на пошту, вказану у профілі користувача. Після відправки буде показано повідомлення - рисунок 3.7.

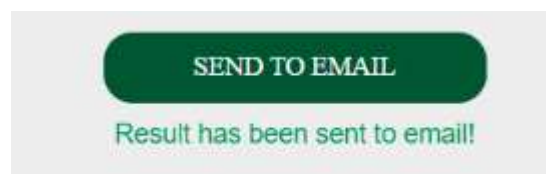


Рисунок 3.7 - Повідомлення відправки на пошту

Результати аналіз зберігаються для користувачів. На рисунку 3.8 зображено профіль користувача із результатами його минулих аналізів.

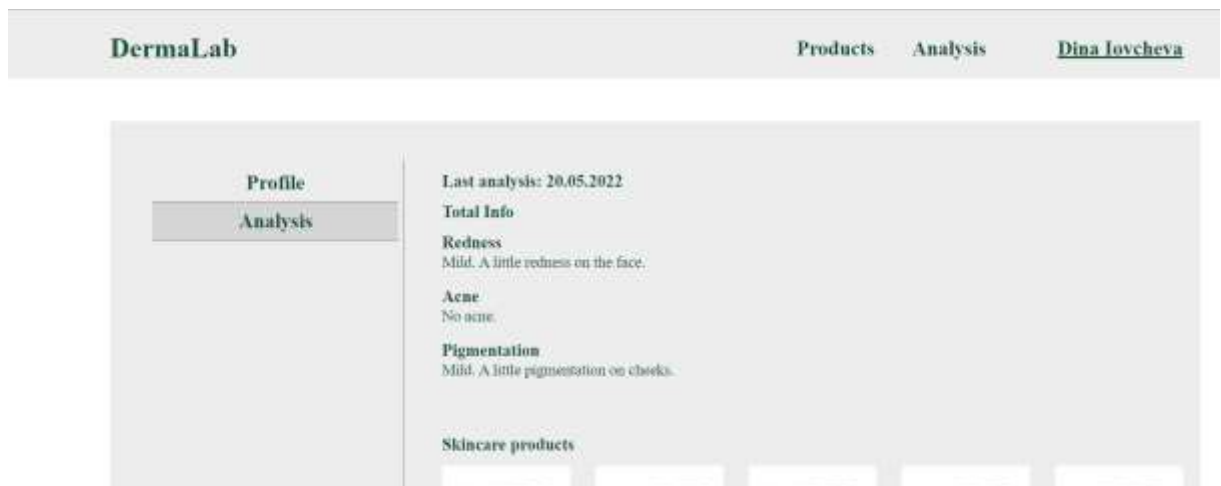


Рисунок 3.8 - Профіль користувача із результатами аналізу

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ РОЗПІЗНАВАННЯ
ВЛАСТИВОСТЕЙ ТА ПРОБЛЕМ ШКІРИ ЛЮДИНИ ЗАСОБАМИ
ШТУЧНОГО ІНТЕЛЕКТУ**

Графічний матеріал

КПІ.IT-8109.045440.05.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олена ХАЛУС

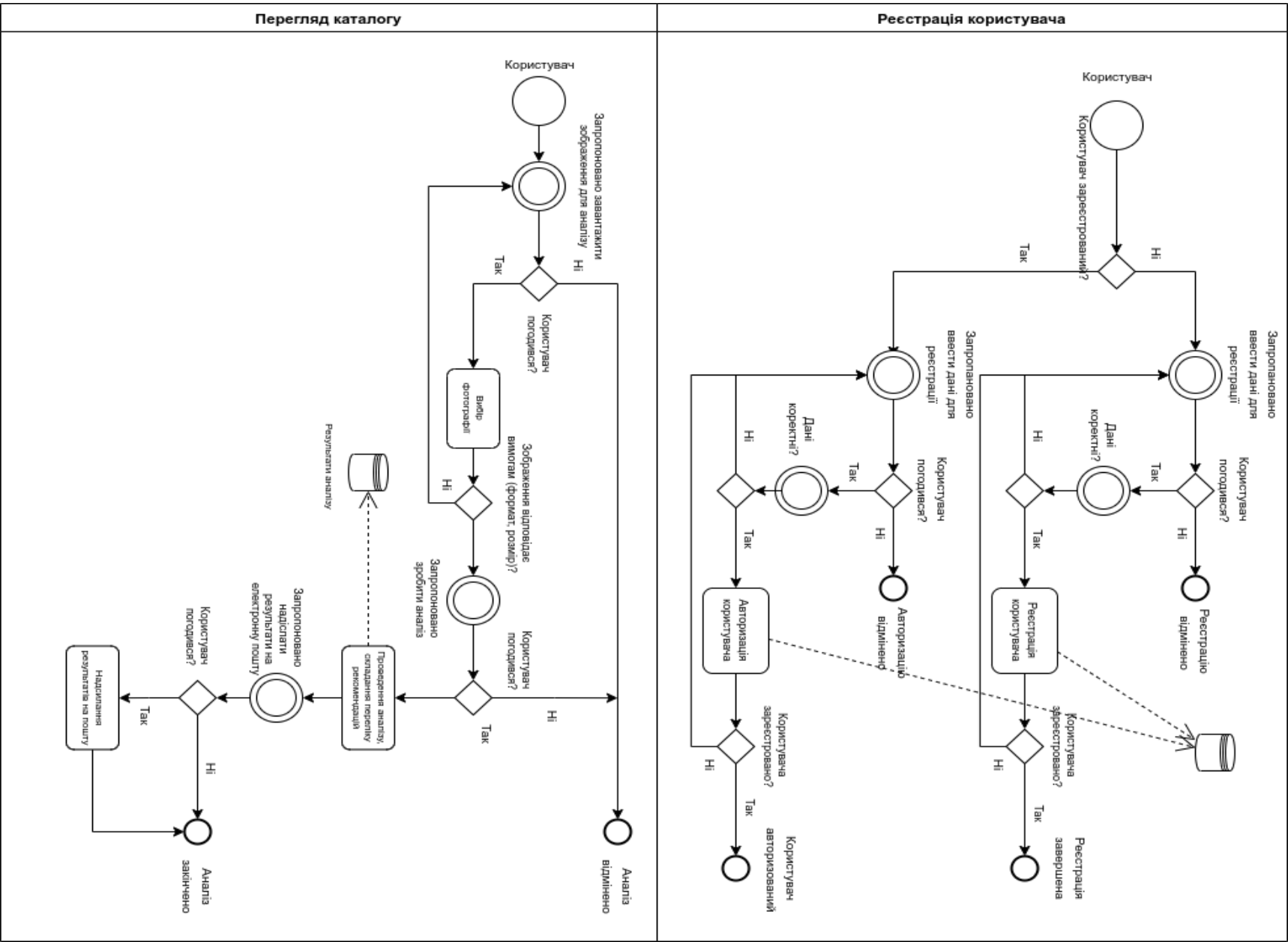
Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

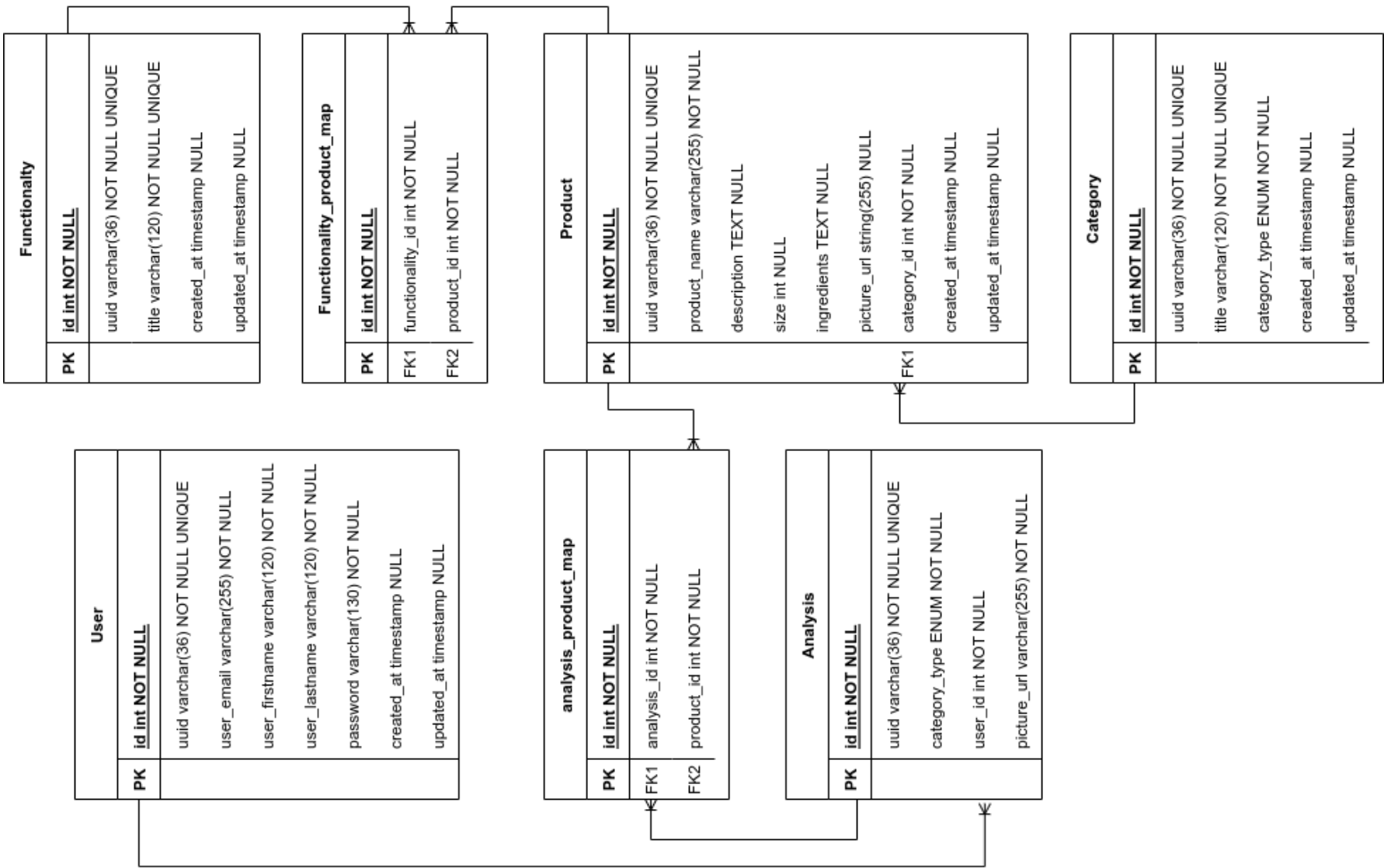
_____ Діна ІОВЧЕВА

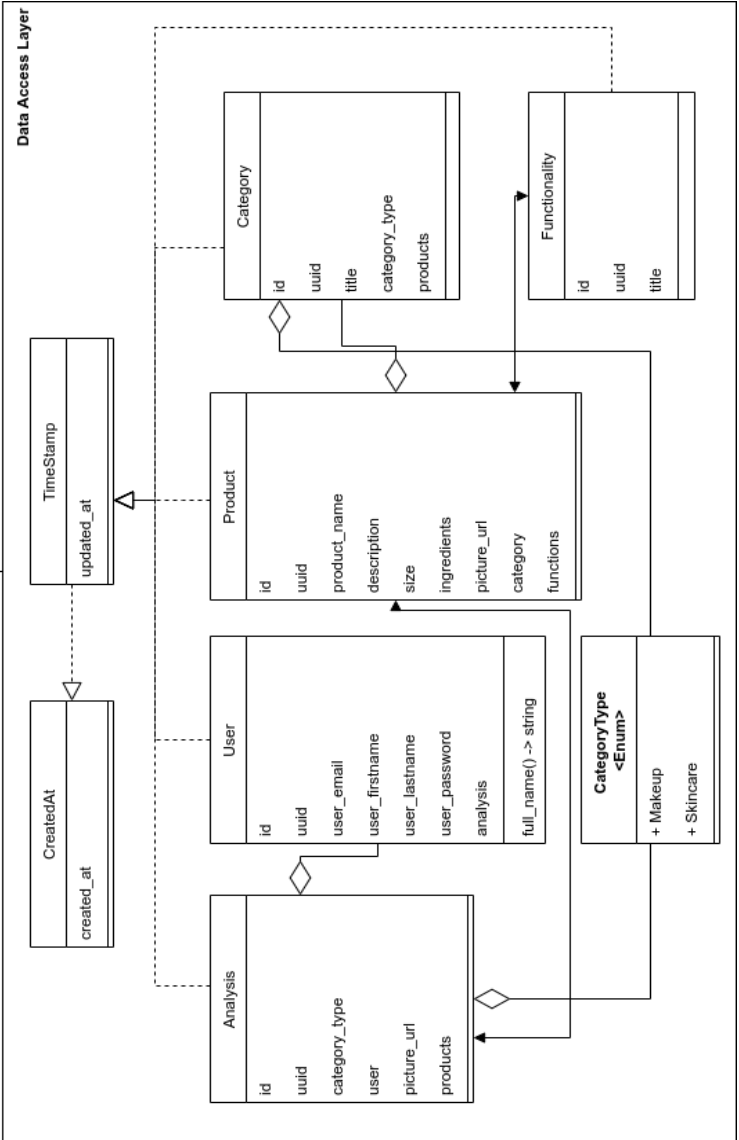
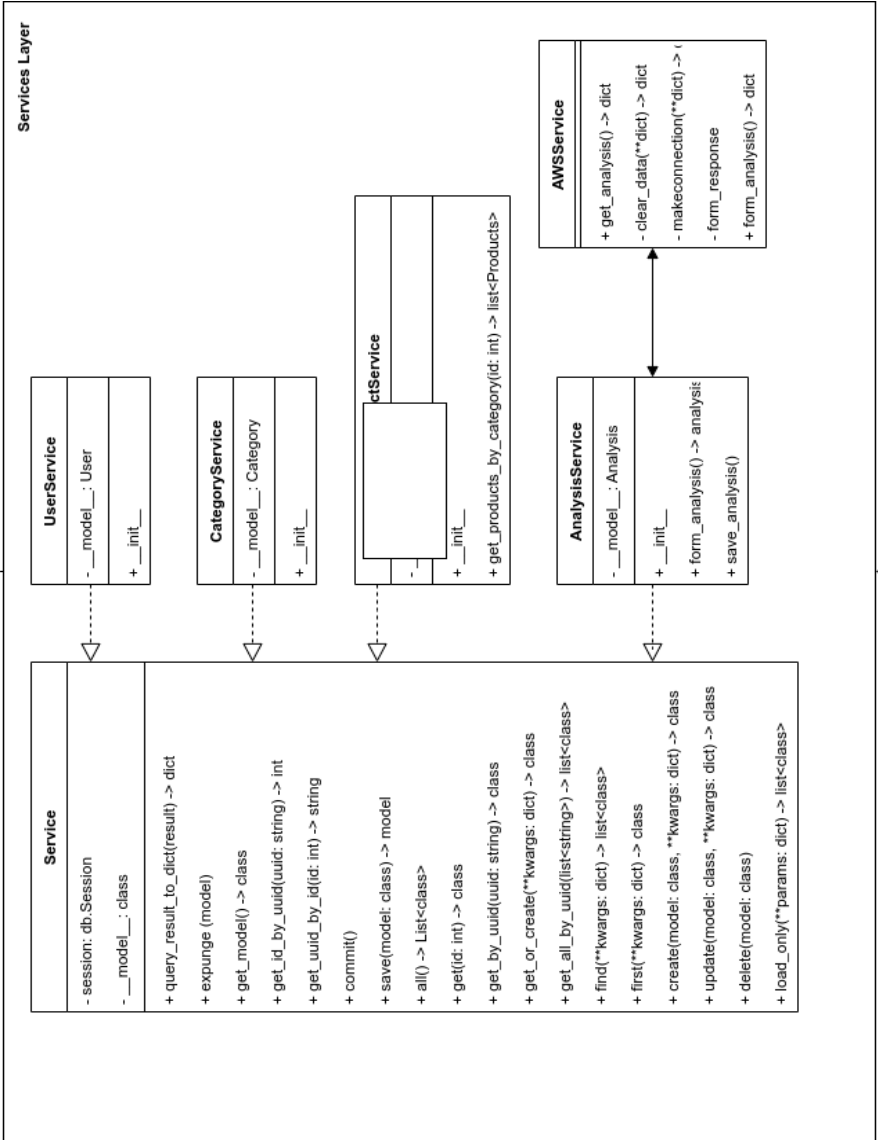
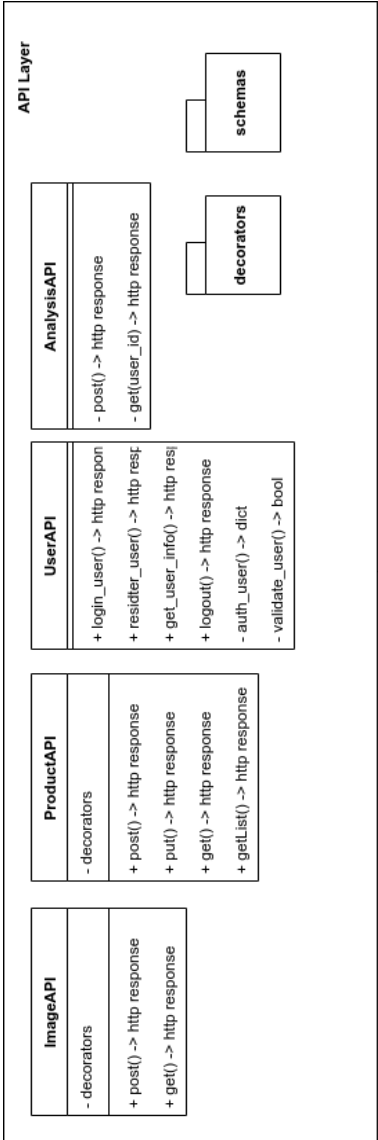
Київ – 2022



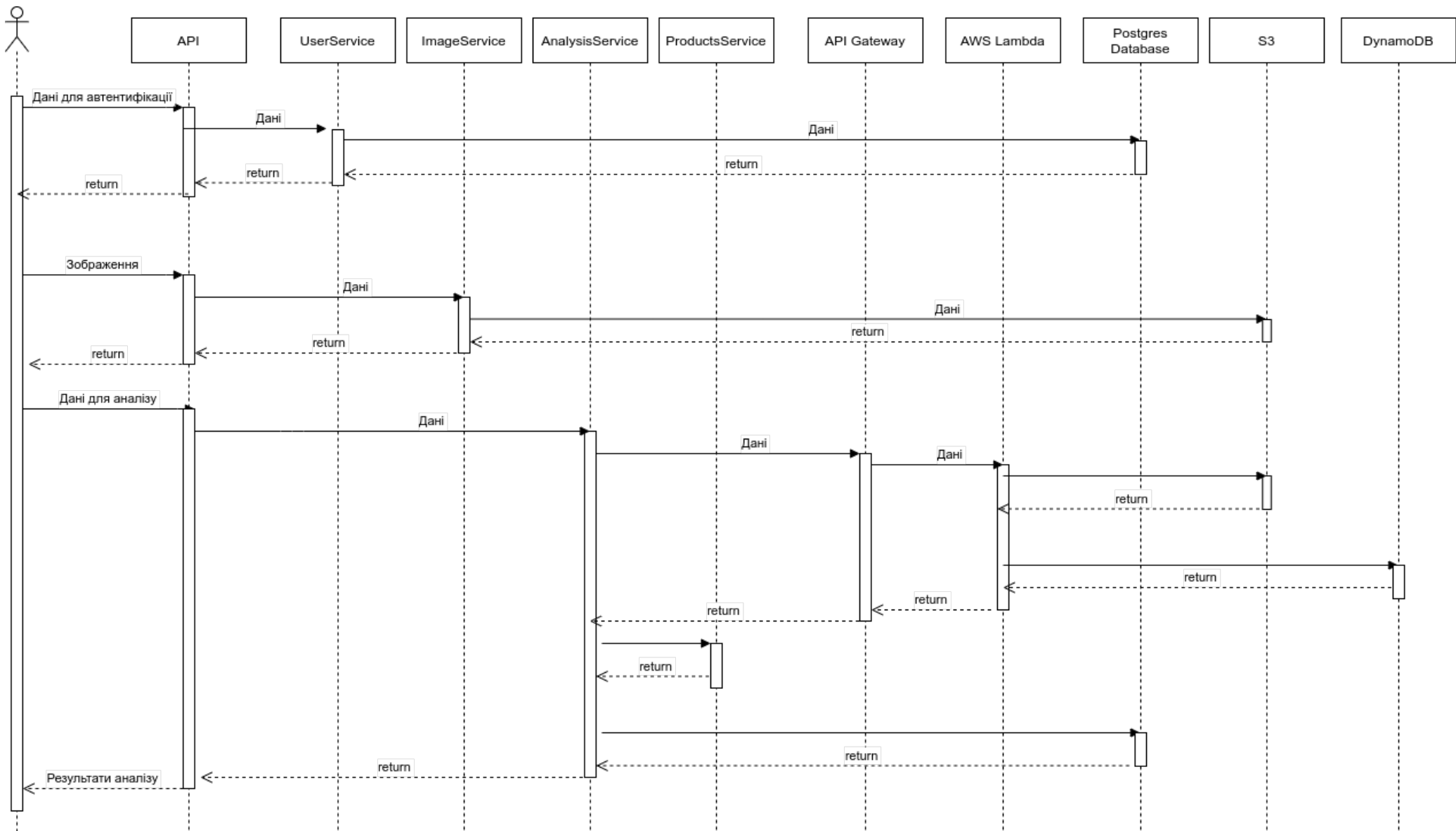
					КПІ.ІТ-8109.045440.06.99								
					Схема структурна бізнес процесів				Літера		Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата					Аркуш 1		Аркушів 1		
Розробив		Іовчева Д.А.											
Перевішив		Халус О.А.											
Т. кон.													
Н. кон.		Ліщук К.І.			Програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелект				КПІ ім. Ігоря Сікорського кафедра ІПІ гр. ІТ-81				
Затвердив		Халус О.А.											

						КПІ.ІТ-8109.045440.07.99					
						Схема бази даних			Літера	Маса	Масштаб
Зм.	Арк.	№ документа	Підпис	Дата							
Розробив		Іовчева Д.А									
Перевірив		Халус О.А.									
Т. кон.									Аркуш 1	Аркушів 1	
Н. кон.		Ліщук К.І.				Програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту			КПІ ім. Ігоря Сікорського кафедра ІПІ гр. ІТ-81		
Затвердив		Халус О.А.									





					КПІ.ІТ-8109.045440.08.99							
					Схема структурна класів	Літера		Маса		Масштаб		
Зм.	Арк.	№ документа	Підпис	Дата	Схема структурна класів	Літера		Маса		Масштаб		
Розробив		Іовчева Д.А.										
Перевішив		Халус О.А.										
Т. кон.												
					Програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелект	Аркуш 1		Аркушів 1				
Н. кон.		Ліщук К.І.				КПІ ім. Ігоря Сікорського кафедра ІПІ гр. ІТ-81						
Затвердив		Халус О.А.										



						КПІ.ІТ-8109.045440.09.99			
						Схема структурна послідовності			
Зм.	Арк.	№ документа	Підпис	Дата					
Розробив		Іовчева Д.А.							
Перевішив		Халус О.А.				Програмне забезпечення для розпізнавання властивостей та проблем шкіри людини засобами штучного інтелекту			
Т. кон.									
Н. кон.		Ліщук К.І.				КПІ ім. Ігоря Сікорського кафедра ІПІ гр. ІТ-81			
Затвердив		Халус О.А.							
						Літера		Маса	Масштаб
						Аркуш 1		Аркушів 1	

Имя пользователя:
Лісовиченко Олег Іванович

Дата проверки:
13.06.2022 10:04:03 EEST

Дата отчета:
13.06.2022 10:04:41 EEST

ID проверки:
1011555905

Тип проверки:
Doc vs Internet + Library

ID пользователя:
76913

Название файла: IT-81_Іовчева_ПЗ

Количество страниц: 49 Количество слов: 7758 Количество символов: 57812 Размер файла: 2.58 MB ID файла: 1011427556

2% Совпадения

Наибольшее совпадение: 0.63% с источником из Библиотеки (ID файла: 12051477)

0.4% Источники из Интернета 3 Страница 51

2% Источники из Библиотеки 150 Страница 51

0% Цитат

Исключение цитат выключено

Исключение списка библиографических ссылок выключено

0% Исключений

Нет исключенных источников